Mathematical Notes on SNARKs

A living technical notebook

August 2026

Contents

1 Overview and Big Picture

A preprocessing SNARK is a proof system for statements of the form

$$\exists w \in \mathbb{F}^m \text{ such that } C(x, w) = 0,$$

where x is public, w is secret, and C is an arithmetic circuit over a finite field. The promise of a SNARK is that the proof is *short*, verification is *fast*, and in the zero-knowledge case the proof reveals essentially nothing beyond the truth of the statement.

These notes unfold in two broad movements. We begin with zero knowledge itself: first in the interactive setting, tracing the idea from cave-and-door intuition through sigma protocols and the Schnorr identification scheme, then crossing to the non-interactive setting via the common reference string. This foundation motivates every design choice in the systems that follow.

We then study SNARKs proper, organized by four principal technical routes through which non-interactive zero-knowledge proofs have been made succinct and practical:

1. **Polynomial IOP + Polynomial Commitment Scheme.** The prover commits to witness polynomials and answers evaluation queries; soundness reduces to the binding property of the commitment. Plonk and Marlin are the canonical examples.
2. **Linear PCP + Bilinear Pairing.** The statement is compiled into a quadratic arithmetic program, encoded as a linear PCP, and verified in a single pairing check. Groth16 and its predecessors (Pinocchio, PGHR13) follow this route.
3. **Folding Schemes for Incrementally Verifiable Computation.** Rather than proving each step independently, the prover *folds* two instances of a relaxed constraint system into one, accumulating computation cheaply and deferring the final SNARK to a single step. Nova is the prototype of this paradigm.
4. **Hash-based IOPs via AIR and FRI.** Computation is described as an algebraic execution trace (AIR), and proximity to the Reed-Solomon code is tested using the FRI protocol. The resulting STARKs are transparent (no trusted setup) and plausibly post-quantum.

A useful slogan is:

Succinct proof for general circuits
= algebraic arithmetization + low-query proof + succinct commitments.

2 What Is Zero Knowledge?

In an ordinary proof you convince someone by showing them the evidence directly — to prove you are over 18, you hand over your ID. The verifier learns your exact date of birth, your name, your address. The proof works, but it reveals far more than necessary.

Zero-knowledge proofs ask: can we do better? Can a prover convince a verifier that a statement is true while revealing *nothing at all* beyond the bare truth of the claim?

The answer, perhaps surprisingly, is yes.

The simulation paradigm

Take *perfect secrecy* as a concrete illustration. A cipher Enc is perfectly secret if

$$\Pr[m \mid c] = \Pr[m]$$

for every message m and every ciphertext c: observing the ciphertext does not shift your beliefs about the plaintext at all. The one-time pad is the canonical example — the ciphertext is uniformly distributed regardless of the message, so it leaks nothing.

There is another way to see why this means no information leaks. Ask: could the adversary have produced the ciphertext themselves, without interacting with the encryption scheme at all? For the one-time pad, the answer is yes — just sample a uniformly random string. The adversary can simulate the ciphertext on their own, so seeing the real one teaches them nothing new. This is the *simulation paradigm*: if you can reproduce the output by yourself, the real output is useless to you.

Exercise. Replace "identical distributions" with "computationally indistinguishable distributions" in the argument above. Write down the resulting security game between an adversary and a challenger, and verify that it captures CPA security for symmetric encryption.

A proof system is **zero knowledge** if there exists an efficient *simulator* that, *without knowing the secret witness*, can produce transcripts that are indistinguishable from real interactions with the honest prover.

The logic is elegant: if the verifier could have generated the same conversation by themselves (by running the simulator), then seeing the real conversation taught them nothing new. Whatever information seems to be in the proof was already accessible to the verifier without the prover's help.

We will develop this intuition through a series of concrete examples.

2.1 The Ali Baba's Cave

2.1.1 Setup

Imagine a circular cave with a single public entrance. Inside, the path splits into two branches, A and B, that meet at a locked door deep in the rock. The door can only be opened with a secret magic word. Alice knows the magic word; Bob does not.

Alice wants to prove to Bob that she knows the magic word — without uttering it.

2.1.2 Protocol

While Bob waits at the entrance, Alice enters the cave and walks down one of the two branches, choosing privately which one. Bob then shouts a challenge from the entrance, naming one of the two sides. Alice must emerge from that side: if she chose the correct branch she simply walks out; otherwise she must pass through the door using the magic word.

Prover P		*Verifier* V
Alice chooses side A or B privately	$\longrightarrow$	
	$\xleftarrow{c}$	$c \leftarrow \{A, B\}$
Alice exits from z, using the word if needed	$\xrightarrow{z}$	Accept iff $c = z$

- Completeness? **Yes**
- Honest-verifier ZK? **Yes**
- Soundness? **Yes**

2.1.3 Analysis

Completeness. If Alice knows the magic word she can always exit from whichever side Bob names, regardless of which side she entered. The verifier always outputs 1.

Honest-verifier zero knowledge. The observable transcript is just (c, z). An honest Alice always has $z = c$, so the transcript contains only the challenge value — which the verifier chose themselves. A simulator that picks $c \leftarrow \{A, B\}$ at random and sets $z = c$ produces the same distribution without knowing any magic word.

Soundness. If Alice does *not* know the magic word, she must commit to one branch before hearing Bob's challenge. She exits successfully only if Bob happens to call the side she already entered. Since Bob's choice is uniformly random, a cheating Alice passes with probability at most $1/2$ per round.

2.1.4 Exercise: The Apple Test

Exercise. Alice has two apples: one red, one green. Bob is red-green colorblind and cannot visually distinguish them. Alice claims she *can* tell them apart. Describe a protocol by which Alice can convince Bob she has the ability to distinguish the apples, without revealing to Bob which apple is red and which is green.

2.2 Sigma Protocols for Discrete Logarithm

We now build a mathematically rigorous protocol for a number-theoretic statement, arriving at the famous *Schnorr identification scheme*. Each attempt below fixes exactly one flaw of the previous one; the sequence of failures makes clear why every design choice in the final protocol is necessary.

2.2.1 Setup

Let $(\mathbb{G}, g, q)$ be a cyclic group of prime order q with generator g. Write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. The *discrete logarithm (DLOG) relation* is

$$\mathcal{R} = \{\, (h,\, w) \mid h = g^w \,\}.$$

The prover P holds a public element $h \in \mathbb{G}$ and a secret witness $w \in \mathbb{Z}_q$ with $h = g^w$. The verifier V holds only h.

The goal: P convinces V that they know w without revealing anything about w itself.

2.2.2 First Attempt: Reveal the Witness Directly

The most direct approach: just send w.

Prover $P(h;\, w)$		*Verifier* $V(h)$
	$\xrightarrow{w}$	Output 1 if $h = g^w$

- Completeness? **Yes**
- Honest-verifier ZK? **No**
- Special soundness? **Yes**

Completeness holds: the verifier checks $h \stackrel{?}{=} g^w$, true by assumption.

Special soundness holds trivially: the response w is itself the witness.

Honest-verifier ZK fails: w is sent in the clear. The verifier directly learns the discrete logarithm.

2.2.3 Second Attempt: Blind the Response with a Nonce

To hide w, the prover randomises the response with a fresh nonce r: commit to g^r and then send r as the response.

Prover $P(h;\, w)$		*Verifier* $V(h)$
$r \leftarrow \mathbb{Z}_q$ $a := g^r$	$\xrightarrow{a}$	
$z := r$	$\xrightarrow{z}$	Output 1 if $g^z = a$

- Completeness? **Yes**
- Honest-verifier ZK? **Yes**
- Special soundness? **No**

Completeness holds: $g^z = g^r = a$.

Honest-verifier ZK holds: $z = r$ is uniform in $\mathbb{Z}_q$ regardless of w. A simulator reproduces the transcript (g^r, r) for any $r \leftarrow \mathbb{Z}_q$ without knowing w.

Soundness fails entirely. The check $g^z \stackrel{?}{=} a$ does not involve h at all. Anyone, regardless of whether they know w, can pick $r \leftarrow \mathbb{Z}_q$, set $a = g^r$ and $z = r$, and the verifier accepts. This protocol proves nothing.

Takeaway. This construction looks stupid, and it is — but deliberately so. The lesson is that zero-knowledge is trivial to achieve if we are willing to abandon soundness entirely: just have the prover send something independent of the witness and the verifier always accept. The hard problem is achieving *both* simultaneously.

2.2.4 Third Attempt: Incorporate the Witness into the Response

The fix: include w in the response via $z := r + w$ and adjust the check to involve h.

Prover $P(h; w)$		*Verifier* $V(h)$
$r \leftarrow \mathbb{Z}_q$ $a := g^r$	$\xrightarrow{a}$	
$z := r + w$	$\xrightarrow{z}$	Output 1 if $g^z = a \cdot h$

- Completeness? **Yes**
- Honest-verifier ZK? **Yes**
- Special soundness? **No**

Completeness holds: $g^z = g^{r+w} = g^r \cdot g^w = a \cdot h$.

Honest-verifier ZK holds: $z = r + w$ is uniform in $\mathbb{Z}_q$ since r is uniform, so the transcript leaks no information about w.

Soundness still fails. A cheating prover can forge a convincing transcript *without knowing* w:

Attack. Choose any $z \leftarrow \mathbb{Z}_q$, then set $a := g^z \cdot h^{-1}$. Send a, then send z. The check becomes $g^z \stackrel{?}{=} a \cdot h = g^z \cdot h^{-1} \cdot h = g^z$. It passes.

2.2.5 What Is Missing?

Notice something about Attempt 3: if the prover *follows the protocol* and genuinely sets $a := g^r$ for a fresh random r, then producing a valid response $z = r + w$ requires knowing w. An honest prover who lacks w is stuck. In this sense, the protocol would be sound — *if* we could trust the prover to generate a correctly.

The problem is that we cannot. The prover is free to choose a in any way they like; the verifier sees only the final value and has no means of checking how it was produced. A cheating prover exploits this by working *backwards*: pick any z first, then set $a := g^z \cdot h^{-1}$ so that the check $g^z = a \cdot h$ passes by construction. No knowledge of w is needed.

Exercise. The prover has complete freedom in choosing a: the verifier sees only the value, not how it was produced. Can you think of a way to modify the protocol so that if the prover deviates from the specification — i.e. does not set $a = g^r$ for some r they know — the verifier has a chance of catching them?

The fix is to insert a verifier challenge *after* the commitment is sent. The challenge creates two competing tests, and the prover must be prepared for either:

- **Challenge** $c = 0$ (protocol compliance): open a by revealing r, i.e. prove $g^z = a$. This checks that the prover actually committed to a valid g^r and knows the exponent.
- **Challenge** $c = 1$ (knowledge of witness): prove $g^z = a \cdot h$. This checks that the prover can incorporate w into the response.

A cheating prover can pre-engineer a to pass one of these tests, but not both simultaneously — because committing to a before the challenge is sent means they cannot adapt it once they learn which check the verifier will run. The challenge forces a genuine commitment.

This is the defining structure of a *sigma protocol*: commit → challenge → respond.

2.2.6 Fourth Attempt: A Binary Challenge

We combine the two verification checks from Attempts 2 and 3, selected by a random bit from the verifier.

Prover $P(h; w)$		*Verifier* $V(h)$
$r \leftarrow \mathbb{Z}_q$ $a := g^r$	$\xrightarrow{a}$	
	$\xleftarrow{c}$	$c \leftarrow \{0, 1\}$
If $c = 0$: $z := r$ If $c = 1$: $z := r + w$	$\xrightarrow{z}$	If $c = 0$, check $g^z = a$; if $c = 1$, check $g^z = ah$

- Completeness? **Yes**
- Honest-verifier ZK? **Yes**
- Special soundness? **Yes**

Soundness error: 1/2.

Completeness: for $c = 0$, $g^z = g^r = a$; for $c = 1$, $g^z = g^{r+w} = a \cdot h$.

Honest-verifier ZK: for each fixed c, a simulator picks $z \leftarrow \mathbb{Z}_q$, sets $a = g^z$ (if $c = 0$) or $a = g^z \cdot h^{-1}$ (if $c = 1$), and outputs (a, c, z). Both distributions match the real transcript.

Why is the simulator allowed to choose a *after* seeing c, while a real prover must commit to a first?

Reason 1 — soundness. If a simulator with no extra power could already fake transcripts, then any cheating prover could run that same strategy and break soundness. The simulator must therefore be given an advantage that a real prover does not have.

Reason 2 — who is the adversary. Zero knowledge asks whether the *verifier* can reproduce the transcript on their own. The verifier is the party trying to learn the witness, so it plays the role of the adversary in the simulation. An honest verifier samples c themselves and therefore already knows c before the transcript is complete. The simulator models exactly this honest verifier, and so is entitled to know c in advance.

This style of argument is called *black-box simulation*: the simulator is given oracle access to the verifier. In the honest-verifier setting this is clean — the verifier's randomness is fixed and used honestly, so the challenge is known to the simulator ahead of time. If the verifier is malicious, it may use its randomness arbitrarily, and the simulator can no longer rely on knowing c in advance. Handling a malicious verifier requires a more powerful technique called *rewinding*, which is beyond our scope here.

Special soundness: given two accepting transcripts $(a, 0, z_0)$ and $(a, 1, z_1)$ with the same a, we have $z_0 = r$ and $z_1 = r + w$. Subtracting:

$$w = z_1 - z_0 \pmod{q}.$$

A cheating prover without w can still succeed with probability $1/2$: they commit to an a valid for one challenge value and hope the verifier asks for it. Running k independent rounds reduces this to 2^{-k}, but there is a cleaner way.

2.2.7 The Schnorr Protocol: Large Challenge Space

Replace the binary challenge with $c \leftarrow \mathbb{Z}_q$. The soundness error drops from $1/2$ to $1/q$, negligible for a cryptographically large prime q.

Prover $P(h; w)$		*Verifier* $V(h)$
$r \leftarrow \mathbb{Z}_q$ $a := g^r$	$\xrightarrow{a}$	
	$\xleftarrow{c}$	$c \leftarrow \mathbb{Z}_q$
$z := r + c \cdot w$	$\xrightarrow{z}$	Output 1 if $g^z = a \cdot h^c$

- Completeness? **Yes**
- Honest-verifier ZK? **Yes**
- Special soundness? **Yes**

Soundness error: $1/q$ (negligible).

Completeness:

$$g^z = g^{r+cw} = g^r \cdot (g^w)^c = a \cdot h^c. \qquad \checkmark$$

Honest-verifier ZK: for any fixed c, the simulator picks $z \leftarrow \mathbb{Z}_q$, sets $a := g^z \cdot h^{-c}$, and outputs (a, c, z). Check: $g^z \stackrel{?}{=} a \cdot h^c = g^z$. The joint distribution of (a, z) is uniform over $\mathbb{G} \times \mathbb{Z}_q$ — identical to the real protocol.

Special soundness: given two accepting transcripts (a, c, z) and (a, c', z') with $c \neq c'$:

$$z = r + cw, \qquad z' = r + c'w \qquad \Longrightarrow \qquad w = \frac{z - z'}{c - c'} \pmod{q}.$$

Division is well-defined because q is prime and $c \neq c'$ implies $c - c'$ is invertible.

Further reading: from identification to signatures and beyond

Identification. The Schnorr protocol is a public-key *identification scheme*: the prover's secret key is $w \leftarrow \mathbb{Z}_q$ and the public key is $h = g^w$. Only whoever knows w can pass the protocol, because computing w from h requires solving the discrete logarithm.

Digital signatures. A signature scheme arises from an identification scheme by removing the verifier: instead of waiting for a random challenge, the signer derives the challenge from the message by hashing, $c = H(a \,\|\, m)$, and then completes the response z as usual. The resulting triple (a, c, z) is a non-interactive proof of knowledge of w that is irrevocably tied to m — any change to the message alters c, invalidating z. This is the **Fiat–Shamir transform** [FS87]; applying it to Schnorr identification yields the **Schnorr signature scheme** [Sch89], standardized as EdDSA [BDL+11]. The same recipe applies to any sigma protocol, giving a post-quantum family: **Falcon** [FHK+20] based on NTRU lattices, **PICNIC** [CDG+17] using only symmetric-key primitives via MPC-in-the-head [IKOS07], and **FAEST** [BBD+23] using VOLE-in-the-head for smaller proofs.

A crucial caveat: the hash function must behave as a *random oracle* for the security proof to work. Without this assumption, non-interactive Fiat–Shamir can be broken in the standard model [GK03].

2.3 Why Can Proofs Be Zero-Knowledge?

Attempt 2 (§2.2.3) made the point starkly: zero knowledge without soundness is trivial — the prover ignores the witness, the verifier ignores h, and simulation is immediate. The hard question is whether we can have *both*. To answer it precisely we need two distinct notions of soundness.

Definition 1 (Soundness)**.** *A proof system is* sound *if for any* false *statement $x \notin \mathcal{L}$, no cheating prover can convince the verifier with more than negligible probability.* Perfect *soundness requires this probability to be exactly zero, even against unbounded provers.*

Definition 2 (Special soundness)**.** *A sigma protocol has* special soundness *if there exists an efficient extractor that, given any two accepting transcripts (a, c, z) and (a, c', z') with the same commitment a but different challenges $c \neq c'$, outputs a valid witness w. Special soundness is the sigma-protocol mechanism underlying an argument of knowledge.*

Definition 3 (Zero knowledge)**.** *A proof system is* zero knowledge *if there exists an efficient simulator that, without the witness, produces transcripts computationally indistinguishable from real ones.*

Remark 1. *The two soundness notions capture different threats. Ordinary soundness rules out provers who try to prove a false statement $x \notin \mathcal{L}$. Knowledge soundness goes further: if a prover convinces the verifier, an extractor can recover a witness for x. The latter is the appropriate notion for a proof of knowledge.*

The Ali Baba cave is an argument — a cheater without the magic word passes with probability at most $1/2$ per round — but not *an argument of knowledge: two accepting transcripts are just a pair of exit choices and carry no algebraic trace of the password from which it could be extracted. Schnorr, by contrast, is an argument of knowledge via special soundness: two transcripts (a, c, z) and (a, c', z') immediately yield $w = (z - z')/(c - c')$.*

Why simulation does not contradict extraction.

The simulator and a real prover operate in different experiments. In honest-verifier Schnorr simulation, the simulator chooses the challenge before constructing the commitment. A real prover must commit first and receives an unpredictable challenge afterwards. In a proof of

knowledge, the extractor has a different advantage: it can obtain two accepting responses to the same commitment under different challenges and then recover the witness.

Neither algorithm is an ordinary online prover. The simulator has the power supplied by the zero-knowledge experiment; the extractor has the power supplied by the knowledge experiment. The real-world prover has neither. Consequently, the existence of a simulator without the witness is compatible with the claim that every successful real-world prover yields an extractable witness.

Remark 2 (Why ordinary soundness is vacuous for the DLOG language)**.** *If $\mathbb{G}$ has prime order and g is a generator, every $h \in \mathbb{G}$ has some discrete logarithm. Thus there are no false group elements outside the language*

$$\mathcal{L}_{\mathrm{DLOG}} = \{h \in \mathbb{G} : \exists w,\ h = g^w\}\,.$$

Ordinary language soundness says little in this case. Schnorr is interesting because it proves knowledge *of the discrete logarithm, which is captured by special soundness rather than by language membership alone.*

2.3.1 The Scope of Zero-Knowledge Proofs

Exercise. A Sudoku puzzle is a 9×9 grid partially filled with digits 1–9, where the goal is to complete the grid so that every row, every column, and every 3×3 box contains each digit exactly once. The completed grid is the *witness*; the partially filled puzzle is the *public statement.*
Design a zero-knowledge proof system that allows a prover to convince a verifier that they know a valid solution to a given Sudoku puzzle, without revealing anything about the solution itself. Your protocol should satisfy completeness, soundness, and zero knowledge. Describe how you would design this system.

The Schnorr protocol proves one specific algebraic statement. A natural question is how broadly zero knowledge applies: is it a niche property of a handful of number-theoretic problems, or does it scale?

Zero knowledge for all of NP. Goldreich, Micali, and Wigderson [GMW86] showed that, assuming one-way functions exist, *every* language in NP has a computational zero-knowledge proof system. The argument proceeds by reduction: any NP statement can be compiled into a graph 3-coloring instance (since 3-coloring is NP-complete), and one can construct a ZK proof for 3-coloring using a commitment scheme. Because the reduction is polynomial-time and ZK composes, this immediately gives a ZK proof for the original statement.

Zero knowledge for all of IP. The scope extends even further. Ben-Or, Goldreich, Goldwasser, Håstad, Kilian, Micali, and Rogaway [BGG+88] showed that *every* language that has an interactive proof — the entire complexity class IP — also has a computational zero-knowledge interactive proof, again assuming one-way functions. Since IP = PSPACE [Sha90], this means zero knowledge reaches well beyond NP into problems believed to be significantly harder.

A necessary assumption. Computational zero knowledge compares the real and simulated views of efficient verifiers. This does not claim that the witness is intrinsically unrecoverable from the public statement. If a verifier can solve the underlying search problem independently—for example, solve the Sudoku directly—then it may learn the witness from the statement, but the proof still reveals no additional efficiently extractable information.

Both general results above require computational assumptions. This is not merely a convenience: the information-theoretic, or perfect/statistical, ZK setting is provably more restricted. Some

non-trivial languages—graph isomorphism is the canonical example—do have perfect ZK proofs. But Fortnow [For87] showed that if every NP language had a perfect ZK proof, the polynomial hierarchy would collapse, which is widely believed not to happen. Computational assumptions are therefore necessary to extend zero knowledge to all of NP and beyond.

2.4 From Interactive to Non-Interactive: The Role of the CRS

2.4.1 The problem with NIZK

In the sigma protocols above, soundness rests on the random challenge c sent by the verifier *after* the prover commits. Recall Attempts 2 and 3: without that challenge, the prover could choose the response first and backwards-engineer a matching commitment. The challenge is the essential ingredient that prevents this.

In a *non-interactive zero-knowledge* (NIZK) proof system there is no back-and-forth. The prover sends a single proof string π; the verifier checks it with no further communication. There is no external c to anchor the commitment.

The difficulty. Without a verifier challenge, the prover and the ZK simulator face exactly the same task: produce π without receiving anything from the outside. If a simulator can generate convincing proofs for true statements without the witness, a cheating prover can attempt the same strategy for false statements. Soundness and zero knowledge appear to be incompatible.

2.4.2 The common reference string

The solution is to introduce shared randomness via a *common reference string* (CRS). A trusted setup algorithm generates

$$(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{SimSetup}(1^\lambda, \mathcal{R}),$$

publishes crs, and discards (or keeps secret) the *trapdoor* td. The crucial asymmetry is:

- A real prover sees only crs and must know the witness w to produce a valid proof.
- The ZK simulator additionally knows the trapdoor td, which lets it produce fake proofs even without w — or even for false statements.

This restores the separation that the interactive challenge provided. In the interactive setting, the simulator's extra power was the ability to see c before committing. In the NIZK setting, the simulator's extra power is the trapdoor td.

3 Foundations of SNARKs

3.1 Why SNARKs exist

Suppose a prover claims to know a huge witness w making a statement true. The trivial proof system is to send w and let the verifier check $C(x, w) = 0$. This is unsatisfactory for three separate reasons:

1. **Privacy:** the witness may be secret.
2. **Communication:** the witness may be much larger than the statement.
3. **Verification cost:** recomputing $C(x, w)$ may be expensive.

SNARKs address all three. The terminology sits in the lineage of succinct arguments and zero-knowledge proofs, beginning with foundational work on zero knowledge, non-interactive zero knowledge, and efficient arguments [GMR89, BFM88, Kil92]. In modern applications this gives:

- private payments and private smart-contract logic,
- validity proofs for rollups and bridges,
- verifiable outsourced computation,
- privacy-preserving compliance and attestations.

3.2 Arithmetic circuits and NP relations

These notes model computation by *arithmetic circuits*. Fix a prime field $\mathbb{F} = \mathbb{F}_p$. An arithmetic circuit is a directed acyclic graph whose internal nodes are labeled by field operations such as $+$, $-$, and $\times$, and whose inputs are field elements.

A circuit with public input $x \in \mathbb{F}^n$ and witness $w \in \mathbb{F}^m$ computes a polynomial-time decidable relation

$$R_C = \{(x, w) : C(x, w) = 0\}.$$

This is the SNARK-friendly analog of an NP relation. The verifier wants evidence that

$$\exists w \text{ such that } (x, w) \in R_C.$$

Example 1 (Circuits as algebraic checks)**.** *Two standard examples are:*

$$C_{\text{hash}}(h, m) = h - \text{SHA256}(m),$$
$$C_{\text{sig}}(pk, m, \sigma) = 0 \quad \textit{iff } \sigma \textit{ verifies under } pk.$$

In practice these are compiled into large arithmetic circuits over some field compatible with the chosen proving system.

3.3 Preprocessing argument systems

A *preprocessing argument system* for a fixed circuit C consists of three algorithms:

$$S(C) \to (\mathsf{pp}, \mathsf{vp}), \qquad P(\mathsf{pp}, x, w) \to \pi, \qquad V(\mathsf{vp}, x, \pi) \to \{0, 1\}.$$

The idea is that S compresses the circuit into verification/proving parameters once and for all. Then many proofs for that circuit can be produced and verified.

Preprocessing is important because verification should depend only polylogarithmically on the circuit size, not linearly. Without preprocessing, the verifier would usually need too much information about the whole circuit to stay succinct.

3.4 Completeness, knowledge soundness, and succinctness

A central security notion for SNARKs is not merely ordinary soundness but *knowledge soundness*. The reason is subtle: we do not merely want "false statements are rejected." We want "if a prover convinces the verifier, then the prover *knows* a valid witness."

Definition 4 (Completeness)**.** *A SNARK is complete if for every valid pair (x, w) with $C(x, w) = 0$,*

$$\Pr\left[V\left(\mathsf{vp}, x, P(\mathsf{pp}, x, w)\right) = 1\right] = 1$$

or at least overwhelmingly close to 1*.*

Definition 5 (Knowledge soundness)**.** *A preprocessing argument system (S, P, V) for C is* knowledge sound *if for every probabilistic polynomial-time adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ there exists a probabilistic polynomial-time extractor E such that, whenever*

$$(\mathsf{pp}, \mathsf{vp}) \leftarrow S(C), \qquad (x, \mathsf{state}) \leftarrow \mathcal{A}_0(\mathsf{pp}), \qquad \pi \leftarrow \mathcal{A}_1(\mathsf{pp}, x, \mathsf{state}),$$

we have

$$\Pr\left[C(x, w) = 0 : w \leftarrow E^{\mathcal{A}_1}(\mathsf{pp}, x, \mathsf{state})\right] \geq \Pr\left[V(\mathsf{vp}, x, \pi) = 1\right] - \epsilon(\lambda),$$

for negligible ϵ.

This definition packages the informal sentence:

> If the verifier accepts with noticeable probability, then an extractor can recover a witness with almost the same probability.

Remark 3 (Why this is stronger than ordinary soundness)**.** *Ordinary soundness only says that a false statement cannot be accepted. Knowledge soundness additionally says that a convincing prover must encode enough information for an extractor to recover a witness. This is exactly the right notion for "proof of knowledge."*

Succinctness means both communication and verification scale sublinearly, ideally polylogarithmically, in the circuit size.

Definition 6 (SNARK)**.** *A SNARK is a preprocessing argument system that is complete, knowledge sound, and succinct. Succinctness informally means*

$$|\pi| = \mathrm{poly}(\lambda, \log|C|, \log|x|) \quad \textit{and} \quad \mathrm{time}(V) = \mathrm{poly}(\lambda, |x|, \log|C|).$$

3.5 Zero knowledge

Zero knowledge was introduced as a way to formalize proofs that reveal no additional knowledge beyond validity of the statement [GMR89]. It is naturally expressed in the standard simulation-based way.

Definition 7 (Zero knowledge, simplified)**.** *A proof system is zero knowledge if there exists an efficient simulator* Sim *such that for every valid public statement x, the simulated transcript is computationally indistinguishable from the real one:*

$$(C, \mathsf{pp}, \mathsf{vp}, x, \pi_{\mathrm{real}}) \approx_c (C, \mathsf{pp}, \mathsf{vp}, x, \pi_{\mathrm{sim}}),$$

where

$$(\mathsf{pp}, \mathsf{vp}) \leftarrow S(C), \qquad \pi_{\mathrm{real}} \leftarrow P(\mathsf{pp}, x, w), \qquad (\mathsf{pp}, \mathsf{vp}, \pi_{\mathrm{sim}}) \leftarrow \mathsf{Sim}(C, x).$$

The key intuition is:

> If the verifier can generate a fake proof transcript by itself, then seeing the real proof did not teach it anything extra about the witness.

In modern systems, zero knowledge is usually achieved by adding random masking polynomials or random blinds to the committed witness polynomials.

3.6 Trusted setup, universal setup, and transparency

It is useful to distinguish three setup models.

1. **Circuit-specific trusted setup.** The setup depends on the exact circuit. Groth16 is the canonical example [Gro16].
2. **Universal (often updatable) trusted setup.** One setup supports many circuits up to some size bound. Sonic and Plonk are canonical examples of this direction [MBKM19, GWC19].
3. **Transparent setup.** No trapdoor secret exists at all. STARKs, IPA/Bulletproof-style systems, and many code-based systems are transparent [BBB+18, BSBHR18b].

The reason trapdoors matter is simple: if the prover learns the hidden setup secret, it may be able to forge proofs for false statements. Ceremony protocols are therefore used for trusted-setup systems: security survives if at least one participant honestly destroys its randomness.

3.7 A quick taxonomy of modern proof systems

The exact performance numbers of proof systems change over time, but the qualitative tradeoffs are robust.

System family	Main strength	Main cost	Setup model
Groth16 [Gro16]	extremely short proofs, very fast verifier	circuit-specific trusted setup	trusted, per circuit
Plonk / Sonic-style systems [GWC19, MBKM19]	universal setup, short proofs, flexible arithmetization	prover slower than Groth16, still not post-quantum	trusted, universal
IPA / Bulletproof-style [BBB+18]	transparent, logarithmic proofs	verifier slower	transparent
STARK / FRI-based [BSBHR18b, BSBHR18a]	transparent, plausibly post-quantum, very scalable prover	large proofs	transparent
DARK / Dory / related [Lee21, BFS20]	transparent logarithmic-size algebraic commitments	heavier algebra, more exotic assump-tions/models	transparent

3.8 From programs to proof systems

The software-stack picture becomes much clearer once one separates front-end programming languages from algebraic constraint systems. Real proving systems do not begin from hand-written field equations. Instead, the workflow is roughly:

$$\text{high-level program} \longrightarrow \text{constraint IR} \longrightarrow \text{prover backend}.$$

Typical high-level languages and DSLs include Circom, ZoKrates, Cairo, Leo, Noir, and others. Earlier systems also studied direct program-execution SNARKs for C and machine-code-style architectures, such as TinyRAM and von Neumann models [BSCG+13, BSCTV14]. The constraint intermediate representations include:

- **R1CS** (Rank-1 Constraint Systems),
- **Plonkish** traces and gate identities,
- **AIR** (used in many STARK systems),
- **PIL** or other polynomial-identity languages.

4 Algebraic Preliminaries

4.1 Finite fields

Most SNARKs arithmetize computation over a finite field. In these notes we usually write

$$\mathbb{F} = \mathbb{F}_p$$

for a prime field. Elements are integers modulo a prime p, and every nonzero element has a multiplicative inverse. The field structure lets arithmetic circuits use addition and multiplication as primitive operations, while the finiteness of the field makes random evaluation arguments meaningful.

The field must be large compared with the degrees of the polynomials that appear in the proof. A typical soundness statement has the form

$$\Pr[\text{bad polynomial identity passes}] \leq \frac{D}{p},$$

where D is a degree bound. Cryptographic systems therefore choose p large enough that this ratio is negligible.

4.2 Univariate polynomials, roots, and degree

A univariate polynomial over $\mathbb{F}$ is

$$f(X) = f_0 + f_1 X + \cdots + f_d X^d.$$

The largest i with $f_i \neq 0$ is the degree. The elementary fact used throughout these notes is:

a nonzero polynomial of degree D has at most D roots.

This is the algebraic reason that checking an identity at a random point can certify a global identity with high probability.

4.3 Evaluation and interpolation

A degree-$< k$ polynomial is uniquely determined by its values at k distinct field points. Given distinct points $a_0, \ldots, a_{k-1}$ and values $y_0, \ldots, y_{k-1}$, the interpolating polynomial is

$$f(X) = \sum_{i=0}^{k-1} y_i \lambda_i(X),$$

where the Lagrange basis polynomial is

$$\lambda_i(X) = \prod_{j \neq i} \frac{X - a_j}{a_i - a_j}.$$

Thus SNARKs can freely switch between coefficient representations and evaluation representations. Plonkish systems strongly prefer the evaluation viewpoint: witness values are placed on a structured domain, then interpolated into low-degree polynomials.

4.4 Roots of unity and subgroup domains

Let $\omega \in \mathbb{F}$ be a primitive k-th root of unity. Then

$$\omega^k = 1, \qquad \omega^i \neq 1 \quad \text{for } 0 < i < k.$$

The associated multiplicative subgroup is

$$\Omega = \{1, \omega, \omega^2, \ldots, \omega^{k-1}\}.$$

Such domains are central because shifting by ω moves to the next row:

$$X \mapsto \omega X.$$

This is exactly what ProductCheck and Plonk gate checks need: local relations between neighboring positions become polynomial identities involving $f(X)$, $f(\omega X)$, and sometimes $f(\omega^2 X)$.

4.5 Vanishing polynomials

For a finite set $\Omega \subseteq \mathbb{F}$, the vanishing polynomial is

$$Z_\Omega(X) = \prod_{a \in \Omega} (X - a).$$

A polynomial f vanishes on Ω iff Z_Ω divides f:

$$f(a) = 0 \ \forall a \in \Omega \quad \Longleftrightarrow \quad f(X) = q(X) Z_\Omega(X).$$

For a multiplicative subgroup $\Omega = \{1, \omega, \ldots, \omega^{k-1}\}$,

$$Z_\Omega(X) = X^k - 1.$$

This special form is why subgroup domains are so efficient: the verifier can evaluate $Z_\Omega(r) = r^k - 1$ in $O(\log k)$ field operations.

For arbitrary subsets, the same theory works with

$$Z_\Omega(X) = \prod_{a \in \Omega} (X - a),$$

but the verifier no longer gets the simple $X^k - 1$ formula unless the subset is itself a subgroup or has some other special structure.

4.6 Polynomial identity testing

The DeMillo–Lipton–Schwartz–Zippel principle says that a nonzero polynomial of total degree D over a field cannot vanish too often [DL78, Sch80, Zip79]. In the univariate case,

$$\Pr_{r \leftarrow \mathbb{F}}[f(r) = 0] \leq \frac{\deg f}{|\mathbb{F}|}$$

for nonzero f. More generally, for a nonzero multivariate polynomial F of total degree D,

$$\Pr_{\mathbf{r} \leftarrow S^m}[F(\mathbf{r}) = 0] \leq \frac{D}{|S|}.$$

This principle is the soundness engine behind polynomial equality testing, ZeroTest, ProductCheck, and permutation arguments.

4.7 Cyclic groups, discrete logarithms, and pairings

Polynomial commitments are cryptographic objects, so they require group assumptions in addition to field algebra. Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order p. In multiplicative notation, every group element has the form g^a for some $a \in \mathbb{F}_p$, and

$$g^a \cdot g^b = g^{a+b}, \qquad (g^a)^b = g^{ab}.$$

The discrete logarithm problem is: given g and $h = g^x$, find x. Pedersen commitments and Bulletproofs-style commitments rely on the hardness of recovering or manipulating hidden linear relations among such exponents [Ped92, BBB+18].

Many pairing-based SNARKs also use a bilinear map

$$e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$$

with

$$e(g^a, g^b) = e(g, g)^{ab}.$$

Pairings let a verifier test multiplicative relations among hidden exponents. A classical cryptographic example is the BLS short-signature construction [BLS01]. KZG and Groth16 both rely on this ability, though in different ways [KZG10, Gro16].

5 Polynomial Commitment Schemes

5.1 The master paradigm: proof system + commitments

The central construction paradigm is:

$$\boxed{\text{Polynomial or oracle proof} + \text{succinct commitment scheme} \Longrightarrow \text{SNARK.}}$$

The commitment scheme lets the prover bind itself to very large algebraic objects – vectors, univariate polynomials, multilinear tables, execution traces – using short commitments. The proof system then only needs to inspect a few locations and to argue that those local checks imply global correctness. This is exactly the architecture behind Plonk: the Plonk layer is a polynomial IOP, and the concrete SNARK depends on which PCS is used to compile the IOP.

A useful separation of responsibilities is:

Layer	Responsibility
Arithmetization / IOP	turn circuit correctness into polynomial identities
PCS	commit to polynomials and prove evaluations succinctly
Fiat-Shamir [FS87]	remove public-coin interaction

5.2 Ordinary commitments and functional commitments

A standard commitment scheme emulates a sealed envelope:

$$\mathsf{Commit}(m; r) \to \mathsf{com}, \qquad \mathsf{Verify}(m, r, \mathsf{com}) \to \{0, 1\}.$$

It should be:

- **binding:** after publishing the commitment, the committer cannot open it to two different messages;

- **hiding:** the commitment leaks no useful information about the message.

A *functional commitment* generalizes this from messages to functions $f \in \mathcal{F}$. The prover commits to a function and later proves evaluations of the committed function at chosen points. Formally, a functional commitment for a function family $\mathcal{F}$ consists of algorithms such as

$$\mathsf{Setup}(\lambda, \mathcal{F}) \to \mathsf{pp},$$

$$\mathsf{Commit}(\mathsf{pp}, f; r) \to \mathsf{com}_f,$$

plus an opening/evaluation protocol proving that

$$\mathsf{com}_f \text{ indeed commits to some } f \in \mathcal{F} \quad \text{and} \quad f(u) = v.$$

5.3 Polynomial commitment schemes

The most important functional commitments for SNARKs are *polynomial commitment schemes* (PCSs). Fix a degree bound d. A PCS for univariate polynomials lets a prover:

1. commit succinctly to $f(X) \in \mathbb{F}[X]$ of degree at most d;
2. later prove succinctly that $f(u) = v$ at a chosen point $u \in \mathbb{F}$.

The syntax is:

$$\mathsf{KeyGen}(\lambda, \mathcal{F}) \to \mathsf{pp},$$

$$\mathsf{Commit}(\mathsf{pp}, f) \to \mathsf{com}_f,$$

$$\mathsf{Eval}(\mathsf{pp}, f, u) \to (v, \pi),$$

$$\mathsf{Verify}(\mathsf{pp}, \mathsf{com}_f, u, v, \pi) \to \{0, 1\}.$$

The ideal complexity target is:

$$\text{opening proof size} = \text{polylog}(d), \qquad \text{verification time} = \text{polylog}(d),$$

with KZG even achieving constant-size openings and constant-time verification in the pairing model [KZG10].

The knowledge-soundness formulation for a PCS says, informally, that if an adversary first outputs a commitment com_f and later successfully opens it at a verifier-chosen point, then an extractor can recover a polynomial $f \in \mathcal{F}$ to which the commitment is bound. This is the PCS-level analogue of SNARK knowledge soundness.

5.4 Algebraic notation for PCS constructions

The algebraic preliminaries above introduced the two notational worlds used below. KZG uses a pairing group and a structured reference string of hidden powers. Bulletproofs-style commitments use independent discrete-log bases and recursive inner-product folding. We will switch between multiplicative notation g^a and additive elliptic-curve notation aG when convenient; they encode the same exponent algebra.

5.5 KZG polynomial commitments

KZG is the canonical constant-size polynomial commitment scheme of Kate, Zaverucha, and Goldberg [KZG10]. It is pairing-based and uses a structured reference string containing powers of a hidden trapdoor τ.

5.5.1 Setup and the hidden trapdoor

Let $\mathbb{G}$ be a cyclic group of prime order p with generator g, and let

$$e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$$

be a bilinear pairing. For degree bound d, the setup samples a secret $\tau \in \mathbb{F}_p$ and publishes

$$\mathsf{pp} = (g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}),$$

then deletes τ.

In additive notation, one writes

$$H_0 = G, \qquad H_1 = \tau G, \qquad H_2 = \tau^2 G, \qquad \ldots, \qquad H_d = \tau^d G.$$

The secret τ must not be known to the prover after setup. If the prover knows τ, it can create fake openings by directly manipulating exponents at the hidden evaluation point.

5.5.2 Commitment in coefficient form

For

$$f(X) = f_0 + f_1 X + \cdots + f_d X^d,$$

KZG defines

$$\mathsf{com}_f = g^{f(\tau)} = g^{f_0} (g^{\tau})^{f_1} \cdots (g^{\tau^d})^{f_d}.$$

In additive notation:

$$\mathsf{com}_f = f(\tau) G = f_0 H_0 + f_1 H_1 + \cdots + f_d H_d.$$

The commitment is deterministic and binding under the relevant pairing assumptions, but plain KZG is not hiding. If zero knowledge is needed, the commitment and opening protocol must be randomized by adding masking terms that are algebraically consistent with the opening equation.

5.5.3 Opening a point by quotienting

To prove that $f(u) = v$, define

$$q(X) = \frac{f(X) - v}{X - u}.$$

This quotient is a polynomial precisely when $v = f(u)$, because $f(u) - v = 0$ means u is a root of $f(X) - v$.

The proof is one group element:

$$\pi = g^{q(\tau)}.$$

The verifier checks

$$e(\mathsf{com}_f / g^v, g) = e(g^{\tau-u}, \pi).$$

Indeed, if $v = f(u)$, then

$$f(X) - v = (X - u)q(X),$$

so evaluating at the hidden point gives

$$f(\tau) - v = (\tau - u)q(\tau).$$

The pairing check is exactly this equality in the exponent:

$$e(g^{f(\tau)-v}, g) = e(g^{\tau-u}, g^{q(\tau)}) = e(g, g)^{(\tau-u)q(\tau)}.$$

In additive notation, the same check is described as wanting

$$(\tau - u) \cdot \mathsf{com}_q \stackrel{?}{=} \mathsf{com}_f - vG,$$

but because the verifier does not know τ, the real implementation uses a pairing to test this hidden-scalar relation.

5.5.4 Why KZG is powerful

KZG has

$$\text{commitment size} = O(1), \qquad \text{opening proof size} = O(1), \qquad \text{verification} = O(1) \text{ pairings},$$

while the prover's commitment and opening work are linear in the degree bound. This is why Plonk with KZG gives very small proofs and very fast verification.

5.6 Security of KZG

5.6.1 Binding and the q-SBDH intuition

A typical computational binding argument uses a pairing-based assumption such as q-Strong Bilinear Diffie-Hellman (q-SBDH). Informally, given

$$(g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}),$$

it should be hard to compute an expression that behaves like

$$e(g, g)^{1/(\tau-u)}$$

for a new u.

Suppose a malicious prover commits to f but successfully opens at u to $v^\star \neq f(u)$. Let

$$\delta = f(u) - v^\star \neq 0.$$

The verification equation implies

$$e(g^{f(\tau)-v^\star}, g) = e(g^{\tau-u}, \pi^\star).$$

Since

$$f(\tau) - v^\star = (\tau - u)q(\tau) + \delta,$$

we can rearrange the equation to isolate a term behaving like

$$e(g, g)^{\delta/(\tau-u)}.$$

Because $\delta \neq 0$, this gives a way to break the q-SBDH-style assumption. This is the algebraic reason a false opening should be infeasible.

5.6.2 Knowledge soundness and the knowledge-of-exponent assumption

Binding says one cannot open a commitment two different ways. Knowledge soundness asks for more: if a prover outputs a commitment, does it actually know the polynomial behind it?

One way to enforce this in KZG is to include a second, independently scaled copy of the SRS. Setup samples α and publishes

$$g^{\alpha}, g^{\alpha\tau}, g^{\alpha\tau^2}, \ldots, g^{\alpha\tau^d}$$

alongside the usual powers of τ. A commitment then consists of

$$\mathsf{com}_f = g^{f(\tau)}, \qquad \mathsf{com}'_f = g^{\alpha f(\tau)}.$$

The verifier checks consistency by

$$e(\mathsf{com}_f, g^{\alpha}) = e(\mathsf{com}'_f, g).$$

The knowledge-of-exponent assumption says that an adversary who can produce such consistent group elements must "know" the coefficients used to form them. In the generic group model, similar conclusions can be argued because the adversary can only compute formal linear combinations of the SRS elements.

5.6.3 Trusted setup ceremonies

A trusted setup is dangerous if one person learns τ. A multi-party ceremony reduces this risk. Suppose a current SRS contains

$$g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}.$$

A participant samples a secret s and updates it to

$$(g^{\tau})^s, (g^{\tau^2})^{s^2}, \ldots, (g^{\tau^d})^{s^d} = g^{\tau s}, g^{(\tau s)^2}, \ldots, g^{(\tau s)^d}.$$

The new trapdoor is τs. If at least one participant samples s honestly and deletes it, nobody knows the final trapdoor. Pairing checks can verify that the updated SRS still consists of consecutive powers.

5.7 Operational optimizations for KZG

5.7.1 Lagrange-basis commitments

If a polynomial is represented by coefficients, computing

$$\mathsf{com}_f = f_0 H_0 + \cdots + f_d H_d$$

is already linear time. But in SNARKs, polynomials are often represented by evaluations

$$f(a_0), f(a_1), \ldots, f(a_d)$$

on an FFT domain. Naively converting these values to coefficients costs $O(d \log d)$.

Let $\lambda_i(X)$ be the Lagrange basis polynomial satisfying

$$\lambda_i(a_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Then

$$f(\tau) = \sum_{i=0}^{d} f(a_i)\lambda_i(\tau).$$

If the SRS is transformed into Lagrange form

$$\widehat{H}_i = \lambda_i(\tau)G,$$

then

$$\mathsf{com}_f = f(a_0)\widehat{H}_0 + \cdots + f(a_d)\widehat{H}_d,$$

again in linear time from the evaluation representation.

5.7.2 Batch openings

For one polynomial opened at many points $u_1, \ldots, u_m$, interpolate the claimed values to a polynomial h of degree less than m satisfying

$$h(u_i) = f(u_i) \quad \text{for all } i.$$

Then

$$f(X) - h(X)$$

vanishes at all u_i, so

$$f(X) - h(X) = \left(\prod_{i=1}^{m}(X - u_i)\right) q(X).$$

The proof is again $g^{q(\tau)}$. For multiple polynomials, one combines the individual quotient polynomials by a random linear combination so that many claims are reduced to one aggregate opening. Multivariate variants and related polynomial-commitment ideas also appear in the PST line of work on signatures of correct computation [PST13].

5.7.3 Vector commitments from PCS

A committed vector $(u_1, \ldots, u_k)$ can be interpreted as the values of a polynomial f on points $1, \ldots, k$. Then a PCS becomes a vector commitment:

1. interpolate f with $f(i) = u_i$;
2. commit to f;
3. open entries by proving $f(i) = u_i$.

In KZG, multiple vector entries can be opened using one group element, often shorter than a Merkle proof, at the cost of pairings and trusted setup [KZG10].

5.8 Bulletproofs and IPA-style polynomial commitments

KZG is extremely succinct but needs a structured trusted setup. Bulletproofs give a transparent alternative based on Pedersen vector commitments and logarithmic inner-product arguments [Ped92, BCC+16, BBB+18]. The price is that verification is no longer constant time.

5.8.1 Pedersen vector commitments

Setup samples independent-looking group generators

$$\mathsf{pp} = (g_0, g_1, \ldots, g_d) \in \mathbb{G}^{d+1}.$$

Here "independent-looking" means no efficient adversary knows discrete-log relations among them. For

$$f(X) = f_0 + f_1 X + \cdots + f_d X^d,$$

define

$$\mathsf{com}_f = \prod_{i=0}^{d} g_i^{f_i}.$$

This is a Pedersen-style vector commitment to the coefficient vector

$$\mathbf{f} = (f_0, \ldots, f_d).$$

The setup is transparent because the generators can be derived by hashing public labels into the group; there is no hidden trapdoor like τ.

5.8.2 Evaluation as an inner product

To open at u, observe that

$$f(u) = \sum_{i=0}^{d} f_i u^i = \langle \mathbf{f}, \mathbf{u} \rangle,$$

where

$$\mathbf{u} = (1, u, u^2, \ldots, u^d).$$

Thus a polynomial opening is an inner-product claim:

$$\langle \mathbf{f}, \mathbf{u} \rangle = v$$

plus the claim that com_f really commits to $\mathbf{f}$.

Bulletproofs recursively reduce the length of this vector relation. Each round halves the dimension while updating the commitment, the bases, and the claimed value. After $O(\log d)$ rounds, the prover reveals one scalar.

5.8.3 The degree-three folding example

A useful core example uses

$$f(X) = f_0 + f_1 X + f_2 X^2 + f_3 X^3$$

and

$$\mathsf{com}_f = g_0^{f_0} g_1^{f_1} g_2^{f_2} g_3^{f_3}.$$

At point u,

$$v = f_0 + f_1 u + f_2 u^2 + f_3 u^3.$$

Split the coefficients into two halves:

$$\mathbf{f}_L = (f_0, f_1), \qquad \mathbf{f}_R = (f_2, f_3).$$

Define

$$v_L = f_0 + f_1 u, \qquad v_R = f_2 + f_3 u.$$

Then

$$v = v_L + u^2 v_R.$$

The prover sends cross commitments

$$L = g_2^{f_0} g_3^{f_1}, \qquad R = g_0^{f_2} g_1^{f_3}.$$

The verifier samples a random nonzero challenge $r \in \mathbb{F}_p^\times$. The folded coefficients are

$$f_0' = r f_0 + f_2, \qquad f_1' = r f_1 + f_3.$$

Intuition for r

The folded claimed value $v' = r v_L + v_R$ can be seen as a one-time MAC, so it perfectly hides the original values v_L and v_R. Also, if the prover tries to cheat by choosing inconsistent halves, the random challenge r ensures that the probability of passing the folded relation is negligible.

The folded bases are

$$g_0' = g_0^{r^{-1}} g_2, \qquad g_1' = g_1^{r^{-1}} g_3.$$

The folded claimed value is

$$v' = r v_L + v_R.$$

Finally, the folded commitment is

$$\mathsf{com}' = L^r \cdot \mathsf{com}_f \cdot R^{r^{-1}}.$$

5.8.4 Algebraic correctness of one folding round

We now verify the key identity. Starting from

$$L^r \mathsf{com}_f R^{r^{-1}} = (g_2^{f_0} g_3^{f_1})^r (g_0^{f_0} g_1^{f_1} g_2^{f_2} g_3^{f_3})(g_0^{f_2} g_1^{f_3})^{r^{-1}},$$

collect the bases:

$$\begin{aligned} \mathsf{com}' &= g_0^{f_0 + r^{-1} f_2} g_1^{f_1 + r^{-1} f_3} g_2^{r f_0 + f_2} g_3^{r f_1 + f_3} \\ &= (g_0^{r^{-1}} g_2)^{r f_0 + f_2} (g_1^{r^{-1}} g_3)^{r f_1 + f_3} \\ &= (g_0')^{f_0'} (g_1')^{f_1'}. \end{aligned}$$

Thus the verifier can replace the old commitment to four coefficients by a new commitment to two folded coefficients.

Similarly, the verifier checks the value relation

$$v \stackrel{?}{=} v_L + u^2 v_R.$$

Then it continues with the folded claim that

$$v' = f_0' + f_1' u.$$

The random challenge r prevents the prover from choosing inconsistent left and right halves that nevertheless pass the folded relation except with small probability.

5.8.5 The general recursive protocol

For length $n = 2^m$, write

$$\mathbf{f} = (\mathbf{f}_L, \mathbf{f}_R), \qquad \mathbf{g} = (\mathbf{g}_L, \mathbf{g}_R).$$

A round sends cross commitments analogous to L and R, receives challenge r, and folds

$$\mathbf{f}' = r\mathbf{f}_L + \mathbf{f}_R,$$

$$\mathbf{g}' = \mathbf{g}_L^{r^{-1}} \circ \mathbf{g}_R,$$

where $\circ$ denotes componentwise multiplication of bases. The evaluation vector is folded compatibly, or in the polynomial-specialized presentation the verifier separately checks the decomposition of v into lower- and higher-degree parts at each round.

After $m = \log_2 n$ rounds, only one scalar remains. The proof contains two group elements per round plus final scalar data, so

$$\text{proof size} = O(\log d).$$

The protocol is public coin and can be made non-interactive using Fiat-Shamir [FS87].

This presentation isolates the commitment-folding algebra used by Bulletproofs-style polynomial openings. A full inner-product argument also folds the evaluation vector and includes the standard inner-product consistency checks; the point here is to expose the part of the protocol that explains the logarithmic opening size.

5.8.6 Complexity and tradeoffs

For Bulletproofs-style PCS:

Operation	Cost
Key generation	$O(d)$ public generators, transparent
Commitment	$O(d)$ group exponentiations
Evaluation proof	$O(d)$ prover work
Proof size	$O(\log d)$ group elements
Verifier time	$O(d)$ in the basic scheme

The essential tradeoff is:

KZG: trusted setup, constant opening

vs.

Bulletproofs: transparent setup, logarithmic opening.

This is why Halo-style systems can avoid trusted setup by using an IPA/Bulletproofs PCS, but the verifier is typically slower than in KZG-based Plonk.

5.9 Other pairing- and discrete-log-based PCS families

Several important schemes sit around KZG and Bulletproofs:

- **Hyrax** arranges coefficients as a two-dimensional matrix, improving verifier time to roughly $O(\sqrt{d})$ while using discrete-log assumptions [WTS+18].
- **Dory** uses pairings and inner pairing-product arguments to delegate structured verifier work to the prover, obtaining $O(\log d)$ proof size and $O(\log d)$ verifier time [Lee21].

- **DARK** uses groups of unknown order to obtain logarithmic proof size and verifier time without a conventional trusted setup [BFS20].
- **FRI/code-based PCS** replace group exponentiations with hashing and low-degree testing [BSBHR18a]. They are transparent and plausibly post-quantum, but proofs are much larger.

Scheme	Setup	Proof size	Verifier	Main assumption
KZG	trusted	$O(1)$	$O(1)$	pairings, q-SBDH-style assumptions
Bulletproofs / IPA	transparent	$O(\log d)$	$O(d)$	discrete logarithm
Hyrax [WTS+18]	transparent	$O(\sqrt{d})$	$O(\sqrt{d})$	discrete logarithm
Dory [Lee21]	transparent	$O(\log d)$	$O(\log d)$	pairings
DARK [BFS20]	transparent	$O(\log d)$	$O(\log d)$	unknown-order groups
FRI / code-based	transparent	larger, often polylogarithmic	hash and query heavy	collision-resistant hashes, codes

6 Polynomial IOPs and the IOP-to-SNARK Paradigm

6.1 The PCP theorem and local verification

For functions $r, q : \mathbb{N} \to \mathbb{N}$, the class $\mathrm{PCP}(r(n), q(n))$ contains languages decided by polynomial-time probabilistic verifiers that use at most $r(n)$ random bits and query at most $q(n)$ symbols of a proof oracle. The identity

$$\mathrm{NP} = \mathrm{PCP}(0, \mathrm{poly}(n))$$

is essentially the definition of NP: one direction reads an ordinary polynomial-size witness in full, while the other records the polynomially many fixed positions read by a deterministic oracle verifier and uses their contents as an NP witness.

The PCP theorem is the much stronger statement

$$\mathrm{NP} = \mathrm{PCP}(O(\log n), O(1)).$$

Completeness can be made 1, while a constant soundness gap can be amplified by repetition. The nontrivial direction constructs a redundant encoding of an NP witness in which a constant number of local consistency checks detect any globally inconsistent proof with constant probability [AS98, ALM+98].

This local-to-global principle is the information-theoretic ancestor of modern IOPs. Kilian's argument makes the cryptographic bridge explicit: commit to a PCP oracle with a Merkle tree, reveal only the verifier's queried symbols and authentication paths, and thereby obtain succinct communication from a locally checkable proof [Kil92]. IOPs retain the oracle viewpoint but allow several rounds of oracle messages and challenges; polynomial IOPs further exploit algebraic structure in those oracles.

6.2 Interactive proofs, IOPs, and polynomial IOPs

To use polynomial commitments inside proof systems, recall the following hierarchy:

- an **interactive proof (IP)** exchanges a small number of messages;
- an **interactive oracle proof (IOP)** allows the prover to send long oracles that the verifier only queries in a few places;
- a **polynomial IOP** is an IOP in which the prover's main oracle messages are polynomials.

A polynomial IOP for circuit satisfiability has the form

$$\mathsf{Setup}(C) \to (\mathsf{pp}, \mathsf{vp}),$$

then the prover sends oracle access to a few polynomials $f_1, \dots, f_t$, the verifier samples random challenges $r_1, \dots, r_s$, and finally checks algebraic identities involving evaluations of those polynomials.

The key point is that the verifier never needs to read entire witness polynomials. It suffices to query them at a few random points.

6.3 From a polynomial IOP to a SNARK

Suppose a polynomial IOP commits to t polynomials and the verifier needs q evaluation proofs. If we instantiate the commitment layer with a PCS, then:

- the prover sends t polynomial commitments;
- each queried evaluation is accompanied by a PCS opening proof;
- the IOP's verifier logic is run on the opened values.

If the IOP is public coin, the Fiat–Shamir transform removes interaction by deriving verifier challenges from a hash of the transcript [FS87]:

$$r_1 = H(C, x, \mathsf{com}_1, \dots), \qquad r_2 = H(C, x, \mathsf{com}_1, \dots, r_1, \dots), \quad \text{etc.}$$

This yields a non-interactive argument in the random-oracle model. Non-interactive zero knowledge in the common-reference-string model originates with Blum, Feldman, and Micali [BFM88]; Fiat–Shamir is a different route, usually modeled through a random oracle.

6.4 Complexity bookkeeping

If the PCS commitment size is pcsCom and the opening size is $\mathsf{pcsOpen}$, then the resulting proof length is roughly

$$t \cdot \mathsf{pcsCom} + q \cdot \mathsf{pcsOpen}.$$

The verifier's work is roughly

$$q \cdot \mathrm{time}(\mathsf{PCS\ verify}) + \mathrm{time}(\text{IOP verifier}),$$

and the prover's work is roughly

$$t \cdot \mathrm{time}(\mathsf{Commit}) + q \cdot \mathrm{time}(\mathsf{Open}) + \mathrm{time}(\text{IOP prover}).$$

This formula explains why the same polynomial IOP can lead to systems with different tradeoffs:

PCS	Resulting flavor	Typical tradeoff
KZG [KZG10]	pairing-based Plonk	short proofs; structured setup
IPA [BBB+18]	IPA-based Plonkish systems	transparent setup; slower verification
FRI [BSBHR18a]	STARK-like systems	transparent and hash-based; larger proofs

7 Univariate Polynomial IOP Gadgets

7.1 Polynomial identity testing: the algebraic engine

The starting point is the Schwartz–Zippel–DeMillo–Lipton lemma [DL78, Sch80, Zip79].

Lemma 1 (Schwartz-Zippel)**.** *Let $f \in \mathbb{F}[X]$ be a nonzero polynomial of degree at most d. If $r \leftarrow \mathbb{F}$ is uniform, then*

$$\Pr[f(r) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

The same statement holds for multivariate polynomials when d is replaced by the total degree.

This turns polynomial equality into random-point testing:

$$f = g \implies f(r) = g(r),$$

and conversely

$$f(r) = g(r) \text{ at a random } r$$

strongly suggests $f = g$ provided the field is large relative to the degree. In common cryptographic parameter regimes one often has $p \approx 2^{256}$ and $d \leq 2^{40}$, so d/p is negligible.

In a compiled proof system, the verifier does not query raw polynomials. Instead, the prover opens committed polynomials at r using the PCS. The IOP verifier checks the algebraic equality on opened values; the PCS verifier checks that these values are consistent with the earlier commitments.

7.2 Using vanishing polynomials in IOP gadgets

The algebraic preliminaries introduced the vanishing polynomial

$$Z_\Omega(X) = \prod_{a \in \Omega} (X - a).$$

The gadgets below repeatedly use the same template: to prove that a polynomial F vanishes on a domain Ω, the prover supplies a quotient q satisfying

$$F(X) = q(X) Z_\Omega(X).$$

For multiplicative subgroup domains this is especially efficient because $Z_\Omega(X) = X^k - 1$. For smaller subdomains, such as the set of gate-start positions in a simplified Plonk trace, one should use the actual vanishing polynomial of that subdomain; it need not be of the form $X^k - 1$.

7.3 ZeroTest

Goal. Given a committed polynomial f, prove

$$f(a) = 0 \qquad \forall a \in \Omega.$$

The prover computes

$$q(X) = \frac{f(X)}{Z_\Omega(X)}.$$

The verifier samples $r \leftarrow \mathbb{F}$ and asks for openings of $f(r)$ and $q(r)$, then checks

$$f(r) \stackrel{?}{=} q(r)Z_\Omega(r).$$

Completeness is immediate. For soundness, if f is not divisible by Z_Ω, then

$$f(X) - q(X)Z_\Omega(X)$$

is a nonzero polynomial of controlled degree. By Schwartz-Zippel, it vanishes at a random r with probability at most degree$/|\mathbb{F}|$.

In compiled SNARK form, q is also committed, and the openings at r are verified by the PCS.

7.4 SumCheck on a subgroup

Goal. Given a committed polynomial f, prove

$$\sum_{a\in\Omega} f(a) = 0.$$

For univariate subgroup domains, a convenient way is to construct an accumulator polynomial s satisfying

$$s(\omega^0) = f(\omega^0),$$

$$s(\omega^i) = \sum_{j=0}^{i} f(\omega^j) \qquad (1 \le i \le k-1).$$

Then

$$\sum_{a\in\Omega} f(a) = 0$$

is equivalent to

$$s(\omega^{k-1}) = 0$$

plus the local recurrence

$$s(\omega X) - s(X) - f(\omega X) = 0 \qquad \text{for all } X \in \Omega.$$

The recurrence is checked by a ZeroTest on the polynomial

$$s(\omega X) - s(X) - f(\omega X).$$

This is the additive analogue of the ProductCheck below.

7.5 ProductCheck

ProductCheck is one of the most important univariate gadgets in Plonk-style systems. It converts a global multiplicative statement into a low-degree recurrence.

The problem statement

Let

$$\Omega = \left\{1, \omega, \omega^2, \ldots, \omega^{k-1}\right\}$$

be a multiplicative subgroup. Given a committed polynomial f, prove

$$\prod_{a \in \Omega} f(a) = 1.$$

Directly multiplying all k values would be expensive for the verifier. Instead, the prover commits to a running product polynomial.

The running product polynomial

Define $t \in \mathbb{F}[X]_{\leq k}$ by its values on Ω:

$$t(1) = f(1),$$

$$t(\omega^s) = \prod_{i=0}^{s} f(\omega^i) \qquad s = 1, \ldots, k-1.$$

Thus

$$t(\omega) = f(1)f(\omega),$$
$$t(\omega^2) = f(1)f(\omega)f(\omega^2),$$

and finally

$$t(\omega^{k-1}) = \prod_{a \in \Omega} f(a).$$

Therefore the claimed product equals 1 if

$$t(\omega^{k-1}) = 1.$$

The local recurrence is

$$t(\omega X) = t(X)f(\omega X) \qquad \forall X \in \Omega.$$

The indexing is worth noticing: the recurrence multiplies by $f(\omega X)$, not $f(X)$, because $t(\omega^s)$ includes the value at ω^s.

The cyclic last step

The final point in the subgroup is $X = \omega^{k-1}$. Since $\omega^k = 1$, the recurrence at this last point reads

$$t(1) = t(\omega^{k-1})f(1).$$

At first this may look circular, because $t(1)$ is also the first running-product value. But this is exactly why the boundary condition is chosen as $t(\omega^{k-1}) = 1$. Under that condition, the last recurrence becomes

$$f(1) = 1 \cdot f(1),$$

so the cyclic wrap-around is consistent. This small indexing point is one of the most common sources of confusion when first learning ProductCheck.

Turning the recurrence into a ZeroTest

Define

$$t_1(X) = t(\omega X) - t(X)f(\omega X).$$

If the recurrence is correct, then

$$t_1(X) = 0 \qquad \forall X \in \Omega.$$

Hence $Z_\Omega(X) = X^k - 1$ divides $t_1(X)$, so the prover can form

$$q(X) = \frac{t_1(X)}{X^k - 1}.$$

The verifier samples $r \leftarrow \mathbb{F}$ and asks for openings of

$$t(\omega^{k-1}), \quad t(r), \quad t(\omega r), \quad q(r), \quad f(\omega r).$$

It accepts if

$$t(\omega^{k-1}) \stackrel{?}{=} 1$$

and

$$t(\omega r) - t(r)f(\omega r) \stackrel{?}{=} q(r)(r^k - 1).$$

This unoptimized ProductCheck protocol uses two new committed polynomials, t and q, and five evaluations. The verifier's non-PCS arithmetic is essentially computing $r^k - 1$, which costs $O(\log k)$ field operations by repeated squaring.

Soundness intuition

If the prover lies about the product, then at least one of two things must happen:

1. the boundary value $t(\omega^{k-1})$ is not 1; or
2. t does not honestly encode the running product, in which case t_1 fails to vanish on Ω.

The second case is caught by the ZeroTest. Since the final check is a random polynomial identity test, the cheating probability is bounded by the relevant degree divided by $|\mathbb{F}|$.

7.6 Rational ProductCheck

The same idea proves product identities for rational functions. Suppose the goal is

$$\prod_{a \in \Omega} \frac{f(a)}{g(a)} = 1,$$

assuming all denominators are nonzero on Ω.

Define

$$t(1) = \frac{f(1)}{g(1)},$$

$$t(\omega^s) = \prod_{i=0}^{s} \frac{f(\omega^i)}{g(\omega^i)} \qquad s = 1, \ldots, k-1.$$

Then the boundary condition is again

$$t(\omega^{k-1}) = 1,$$

and the recurrence becomes

$$t(\omega X)g(\omega X) = t(X)f(\omega X) \qquad \forall X \in \Omega.$$

Define

$$t_1(X) = t(\omega X)g(\omega X) - t(X)f(\omega X).$$

Then run ZeroTest on t_1 over Ω.

This rational version is the bridge from ProductCheck to permutation and wiring arguments.

In randomized permutation arguments, the denominators depend on verifier challenges. A small set of "bad" challenges may make a denominator vanish. Protocol descriptions usually exclude those challenges or absorb their probability into the usual Schwartz–Zippel-style soundness error.

7.7 PermutationCheck via randomized products

Goal. Given committed polynomials f and g, prove that the two lists

$$(f(1), f(\omega), \ldots, f(\omega^{k-1}))$$

and

$$(g(1), g(\omega), \ldots, g(\omega^{k-1}))$$

are the same multiset.

Define the characteristic polynomials of the two multisets:

$$\widehat{f}(Z) = \prod_{a \in \Omega} (Z - f(a)),$$

$$\widehat{g}(Z) = \prod_{a \in \Omega} (Z - g(a)).$$

The two lists are permutations of each other iff

$$\widehat{f}(Z) = \widehat{g}(Z).$$

By Schwartz-Zippel, it suffices to choose random $r \leftarrow \mathbb{F}$ and prove

$$\widehat{f}(r) = \widehat{g}(r).$$

Equivalently,

$$\prod_{a \in \Omega} \frac{r - f(a)}{r - g(a)} = 1.$$

This is exactly a rational ProductCheck with

$$F(a) = r - f(a), \qquad G(a) = r - g(a).$$

This technique is often called Lipton's trick: equality of large multisets is reduced to equality of two random products. Closely related product and shuffle arguments appear in efficient zero-knowledge shuffle proofs [BG12].

7.8 Prescribed permutation checks

The previous gadget proves that two lists are equal up to some unknown permutation. Plonk wiring constraints need more: the permutation is prescribed by the circuit.

Let

$$W : \Omega \to \Omega$$

be a known permutation. In protocols it is often encoded by its own low-degree interpolation polynomial on Ω, but that polynomial should not be interpreted as a cheap function to compose inside $g(W(X))$. Given committed polynomials f and g, the goal is

$$f(y) = g(W(y)) \qquad \forall y \in \Omega.$$

A direct ZeroTest of

$$f(X) - g(W(X))$$

is not efficient, because the composition $g(W(X))$ can have degree about k^2 even when f, g, and W individually have degree $O(k)$. That would destroy the linear/quasilinear prover goal.

The trick is to encode pairs. Consider the two multisets

$$\{(W(a), f(a)) : a \in \Omega\}$$

and

$$\{(a, g(a)) : a \in \Omega\}.$$

The reason for introducing pairs is conceptual: the first coordinate identifies a wire slot, while the second coordinate records the value stored in that slot. Equality of the pair multisets says that every destination slot receives exactly the value it should receive.

If these multisets are equal, then the value attached to $W(a)$ on the left matches the value attached to the same point on the right, which implies

$$f(a) = g(W(a))$$

for every $a \in \Omega$.

To test equality of pair-multisets, define bivariate polynomials

$$\widehat{F}(X, Y) = \prod_{a \in \Omega} \big(X - YW(a) - f(a)\big),$$

$$\widehat{G}(X, Y) = \prod_{a \in \Omega} \big(X - Ya - g(a)\big).$$

Because $\mathbb{F}[X, Y]$ is a unique factorization domain, equality of these products of linear factors is equivalent to equality of the corresponding multisets of pairs.

The verifier samples random $r, s \leftarrow \mathbb{F}$ and asks the prover to prove

$$\widehat{F}(r, s) = \widehat{G}(r, s).$$

This is equivalent to the rational product identity

$$\prod_{a \in \Omega} \frac{r - sW(a) - f(a)}{r - sa - g(a)} = 1.$$

Again, a rational ProductCheck proves the identity without explicitly constructing the degree-k bivariate products.

8 The Plonk Polynomial IOP

8.1 A pedagogical one-column Plonk arithmetization

This section presents a pedagogical one-column version of Plonk. Real Plonkish systems usually use several witness columns and row-wise gate identities; the one-column presentation below is meant to make the algebraic dependencies visible before introducing production-level notation.

Plonk can be presented as a polynomial IOP for arithmetic circuits. Consider the circuit

$$(x_1 + x_2)(x_2 + w_1).$$

With input values

$$x_1 = 5, \qquad x_2 = 6, \qquad w_1 = 1,$$

the three gates are:

Gate	left input	right input	output
0	5	6	11
1	6	1	7
2	11	7	77

Gate 0 and gate 1 are additions. Gate 2 is a multiplication.

The simplified encoding uses one trace polynomial T. Let $|C|$ be the number of gates and $|I| = |I_x| + |I_w|$ the total number of public and witness inputs. Define

$$d = 3|C| + |I|.$$

Choose a subgroup

$$\Omega = \left\{1, \omega, \omega^2, \ldots, \omega^{d-1}\right\}.$$

The trace polynomial $T \in \mathbb{F}[X]_{\leq d}$ is defined by:

$$T(\omega^{-j}) = \text{input number } j \qquad j = 1, \ldots, |I|,$$

and, for gate $\ell = 0, \ldots, |C| - 1$,

$$T(\omega^{3\ell}) = \text{left input of gate } \ell,$$
$$T(\omega^{3\ell+1}) = \text{right input of gate } \ell,$$
$$T(\omega^{3\ell+2}) = \text{output of gate } \ell.$$

Since these values are specified on a domain, the prover can interpolate T by FFT/NTT methods.

8.2 The four conditions that make a valid trace

The prover commits to T. The verifier must be convinced of four facts.

(1) Public input correctness

Let $v(X)$ be the low-degree polynomial interpolated by the verifier from the public inputs. Let

$$\Omega_{\text{inp}} = \left\{\omega^{-1}, \ldots, \omega^{-|I_x|}\right\}.$$

The public input check is

$$T(y) - v(y) = 0 \qquad \forall y \in \Omega_{\text{inp}}.$$

This is a ZeroTest over the input subset.

(2) Gate correctness

Introduce a selector polynomial $S(X)$ such that, at each gate-start point $y = \omega^{3\ell}$,

$$S(y) = \begin{cases} 1, & \text{if gate } \ell \text{ is addition,} \\ 0, & \text{if gate } \ell \text{ is multiplication.} \end{cases}$$

Let

$$\Omega_{\text{gates}} = \left\{1, \omega^3, \omega^6, \ldots, \omega^{3(|C|-1)}\right\}.$$

For every $y \in \Omega_{\text{gates}}$, the gate equation is

$$S(y)(T(y) + T(\omega y)) + (1 - S(y))T(y)T(\omega y) - T(\omega^2 y) = 0.$$

If $S(y) = 1$, this says left plus right equals output. If $S(y) = 0$, it says left times right equals output.

(3) Wiring correctness

The circuit imposes copy constraints. For example, the same value $x_2 = 6$ may appear as a public input, as the right input of gate 0, and as the left input of gate 1. These equalities are encoded by a permutation

$$W : \Omega \to \Omega$$

whose cycles connect slots that must hold the same value. The logical wiring condition is

$$T(y) = T(W(y)) \qquad \forall y \in \Omega.$$

It is proved using the prescribed permutation check above, not by directly composing $T(W(X))$.

(4) Output correctness

If the convention is that the circuit output should be zero, then the final gate output position is checked as

$$T(\omega^{3|C|-1}) = 0.$$

If the desired output is a public value y_{out}, one can either check

$$T(\omega^{3|C|-1}) = y_{\text{out}}$$

directly by a PCS opening, or append a final subtraction gate so that the zero-output convention applies.

8.3 The complete Plonk Poly-IOP

The setup for the simplified IOP outputs the circuit-dependent polynomials

$$S, \quad W,$$

where S encodes gate types and W encodes wiring. The prover constructs and commits to T. It then proves:

1. $T(y) - v(y) = 0$ on Ω_{inp};

2. the gate polynomial

$$S(X)(T(X)+T(\omega X))+(1-S(X))T(X)T(\omega X)-T(\omega^2 X)$$

   vanishes on Ω_{gates};

3. wiring correctness by the prescribed permutation ProductCheck;

4. output correctness at the final output point.

In the full Plonk analysis, the resulting Poly-IOP is complete and knowledge sound under a degree-to-field-size condition such as $7|C|/p$ being negligible [GWC19]. The exact constant is less important than the principle: all possible cheating strategies are reduced to nonzero polynomial identities that would have to vanish at random points.

When this public-coin IOP is compiled with a PCS and Fiat-Shamir, the result is a SNARK [FS87].

8.4 Why this is efficient

The design is efficient for three mutually reinforcing reasons:

1. the whole witness is compressed into a few low-degree polynomials;

2. global constraints are reduced to polynomial identities on subgroups;

3. those identities are checked at a few random points, not everywhere.

The heavy lifting is shifted to fast polynomial arithmetic (FFT/NTT, interpolation, quotient computation) and succinct opening proofs.

8.5 From simplified Plonk to modern Plonkish systems

The simplified one-column trace above is only a teaching model. Real Plonkish systems are more expressive.

- They usually use multiple witness columns rather than a single trace polynomial.
- They allow **custom gates**, so one row can enforce an identity more general than a pure $+$ or $\times$ gate.
- They use **lookup arguments**, for example Plookup [GW20], to prove that values belong to fixed tables.
- They often use a **grand product polynomial** to implement permutation/copy constraints more compactly.
- They can be paired with KZG, IPA/Bulletproofs, or FRI to obtain different concrete SNARK systems.

A representative custom gate has the form

$$v(\omega y)+w(y)t(y)-t(\omega y)=0 \qquad \forall y\in\Omega_{\text{gates}}.$$

Such identities are simply added to the gate-check polynomial.

HyperPlonk-style systems replace the univariate subgroup domain by the Boolean hypercube $\{0,1\}^t$ and replace univariate ZeroTests by multilinear SumCheck protocols. The motivation is to reduce prover time by avoiding some expensive univariate quotient computations.

8.6 Choice of PCS determines the concrete SNARK

One of the deepest practical lessons is that the same underlying Plonk IOP can be combined with different commitment layers:

- **Plonk + KZG** gives short proofs and very fast verification, but needs a trusted setup [KZG10, GWC19].
- **Plonk + IPA PCS** gives transparency but slower verification [BBB+18].
- **Plonk + FRI** gives transparency and plausible post-quantum security, but larger proofs [BSBHR18a].

This explains why "Plonk" in practice is best thought of as an *arithmetization / IOP layer*, not as one single monolithic proving system.

9 Groth16 and Pairing-Based SNARKs

The previous chapters developed the Plonk route to SNARKs: computation is represented as a trace, local constraints become polynomial identities, and wiring is enforced by a permutation argument. Groth16 follows a different but equally central route. It starts from a rank-1 constraint system (R1CS), converts that system into a quadratic arithmetic program (QAP), and proves the resulting divisibility relation with a preprocessing pairing argument [GGPR13, PHGR13, Gro16].

For a fixed circuit, Groth16 has perfect completeness, perfect zero knowledge, a proof consisting of two elements of $\mathbb{G}_1$ and one element of $\mathbb{G}_2$, and a verifier whose pairing cost is independent of the circuit size. Its succinctness comes at the price of a circuit-specific structured reference string (SRS): the setup randomness must be sampled correctly and then destroyed.

9.1 The arithmetization pipeline

The Groth16 route has the following shape:

$$\boxed{\text{arithmetic circuit} \longrightarrow \text{R1CS matrices} \longrightarrow \text{QAP divisibility} \longrightarrow \text{pairing equation}}.$$

The conceptual transformation is

$$\begin{aligned} &\text{the circuit is evaluated correctly} \\ \iff\quad &\text{all rank-1 constraints are satisfied} \\ \iff\quad &P_z(r) = 0 \text{ for every constraint point } r \in H \\ \iff\quad &Z_H(X) \mid P_z(X). \end{aligned}$$

Thus many gate equations become one polynomial identity.

9.2 From algebraic circuits to R1CS

Let $\mathbb{F} = \mathbb{F}_p$ be a prime field. A satisfying assignment is written

$$z = (a_0, a_1, \ldots, a_m) \in \mathbb{F}^{m+1}, \qquad a_0 = 1.$$

The public coordinates are $(a_0, \ldots, a_\ell)$, and the private witness coordinates are $(a_{\ell+1}, \ldots, a_m)$. An R1CS instance with n constraints consists of matrices

$$A, B, C \in \mathbb{F}^{n \times (m+1)}.$$

Its qth constraint is

$$\left(\sum_{i=0}^{m} A_{q,i} a_i\right)\left(\sum_{i=0}^{m} B_{q,i} a_i\right) = \sum_{i=0}^{m} C_{q,i} a_i.$$

Equivalently,

$$(Az) \circ (Bz) = Cz,$$

where $\circ$ denotes component-wise multiplication. Addition gates fit this form by multiplying a linear form by the constant wire $a_0 = 1$; multiplication gates already have the required rank-1 form. Appendix C gives a color-coded example that tracks every matrix column into the corresponding QAP polynomial.

9.3 From R1CS to QAP

Choose distinct constraint points

$$H = \{r_1, \ldots, r_n\} \subset \mathbb{F}, \qquad t(X) = Z_H(X) = \prod_{q=1}^{n} (X - r_q).$$

For every variable index i, interpolate polynomials $u_i, v_i, w_i \in \mathbb{F}[X]$ of degree at most $n-1$ satisfying

$$u_i(r_q) = A_{q,i}, \qquad v_i(r_q) = B_{q,i}, \qquad w_i(r_q) = C_{q,i}.$$

The ith columns of A, B, C therefore become u_i, v_i, w_i. Aggregate them using the assignment:

$$U(X) = \sum_{i=0}^{m} a_i u_i(X), \qquad V(X) = \sum_{i=0}^{m} a_i v_i(X), \qquad W(X) = \sum_{i=0}^{m} a_i w_i(X).$$

At r_q these polynomials evaluate to the three linear forms in the qth R1CS row. Hence all constraints hold if and only if

$$P_z(r_q) = U(r_q)V(r_q) - W(r_q) = 0 \qquad (q = 1, \ldots, n).$$

By the vanishing-polynomial criterion, this is equivalent to the existence of a polynomial $h(X)$ such that

$$U(X)V(X) - W(X) = h(X)t(X).$$

Since $\deg U, \deg V, \deg W \le n-1$, one may take $\deg h \le n-2$. The QAP relation compresses all n R1CS equations into one divisibility claim.

9.4 Pairing notation

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order p, and let

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \longrightarrow \mathbb{G}_T$$

be a non-degenerate asymmetric bilinear pairing. Write

$$[x]_1 = g_1^x, \qquad [x]_2 = g_2^x, \qquad [x]_T = e(g_1, g_2)^x.$$

Group operations in the source groups add exponents, while the pairing multiplies them:

$$[x]_j + [y]_j = [x + y]_j, qquadc[x]_j = [cx]_j, qquade([x]_1, [y]_2) = [xy]_T.$$

The additive notation on $\mathbb{G}_1$ and $\mathbb{G}_2$ is only typographical; the target-group equation below is multiplicative.

9.5 The structured reference string

Setup samples

$$(\alpha, \beta, \gamma, \delta, \tau) \xleftarrow{\$} \mathbb{F}^5, \qquad \gamma, \delta \neq 0.$$

For each public coordinate $0 \leq i \leq \ell$, define

$$K_i(\tau) = \frac{\beta u_i(\tau) + \alpha v_i(\tau) + w_i(\tau)}{\gamma},$$

and for each private coordinate $\ell < i \leq m$, define

$$L_i(\tau) = \frac{\beta u_i(\tau) + \alpha v_i(\tau) + w_i(\tau)}{\delta}.$$

The proving key contains the source-group encodings needed to form $[\alpha + U(\tau)]_1$, $[\beta + V(\tau)]_2$, the private-input linear combination $\sum_{i>\ell} a_i [L_i(\tau)]_1$, and

$$\left\{ \left[\frac{\tau^j t(\tau)}{\delta} \right]_1 \right\}_{j=0}^{n-2}.$$

Concretely this includes suitable encodings of the $u_i(\tau)$ values in $\mathbb{G}_1$, the $v_i(\tau)$ values in $\mathbb{G}_2$ (and the auxiliary encodings required to form the cross terms in C), together with $[\delta]_1$ and $[\delta]_2$. The verification key contains

$$[\alpha]_1, quad[\beta]_2, quad[\gamma]_2, quad[\delta]_2, \quad \{[K_i(\tau)]_1\}_{i=0}^{\ell}.$$

The exact published tuple is circuit dependent. Security requires that no party retain the toxic waste $(\alpha, \beta, \gamma, \delta, \tau)$ after setup.

The scalars have separate algebraic roles. The hidden point τ binds the proof to one QAP identity; α and β tie the two multiplicands to the same assignment; γ isolates the verifier-controlled public-input contribution; and δ isolates the witness-dependent part and supports zero-knowledge randomization.

9.6 Proving

Given a satisfying assignment and the quotient $h(X)$, the prover samples independent $r, s \xleftarrow{\$} \mathbb{F}$ and forms the exponent-level quantities

$$\begin{aligned} A &= \alpha + U(\tau) + r\delta, \\ B &= \beta + V(\tau) + s\delta, \\ C &= \frac{\sum_{i=\ell+1}^{m} a_i\big(\beta u_i(\tau) + \alpha v_i(\tau) + w_i(\tau)\big) + h(\tau)t(\tau)}{\delta} + sA + rB - rs\delta. \end{aligned}$$

The actual proof exposes only group encodings:

$$\pi = ([A]_1, [B]_2, [C]_1) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1.$$

Every term is computed as a linear combination of proving-key elements; the prover never learns the setup scalars.

9.7 Verification and completeness

From the public coordinates, the verifier computes

$$[K_{\text{pub}}]_1 = \sum_{i=0}^{\ell} a_i [K_i(\tau)]_1.$$

It accepts exactly when

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2)\, e([K_{\text{pub}}]_1, [\gamma]_2)\, e([C]_1, [\delta]_2).$$

The first pairing may be precomputed as $[\alpha\beta]_T$. Thus online verification uses one pairing-product equation with three pairings, plus a public-input multiscalar multiplication.

To prove completeness, expand the left exponent:

$$\begin{aligned} AB &= (\alpha + U + r\delta)(\beta + V + s\delta) \\ &= \alpha\beta + \beta U + \alpha V + UV + s\delta(\alpha + U) + r\delta(\beta + V) + rs\delta^2, \end{aligned}$$

where all polynomials are evaluated at τ. The QAP identity $UV = W + ht$ gives

$$\beta U + \alpha V + UV = \sum_{i=0}^{m} a_i(\beta u_i + \alpha v_i + w_i) + ht.$$

On the other hand,

$$\begin{aligned} \gamma K_{\text{pub}} + \delta C = \sum_{i=0}^{m} a_i(\beta u_i + \alpha v_i + w_i) + ht \\ + s\delta A + r\delta B - rs\delta^2. \end{aligned}$$

Substituting A and B into the last line yields precisely all remaining terms of AB. Therefore

$$AB = \alpha\beta + \gamma K_{\text{pub}} + \delta C,$$

and bilinearity gives the verifier's pairing equation.

9.8 Zero knowledge and knowledge soundness

For fixed $\delta \neq 0$, the randomizers make A and B uniform over $\mathbb{F}$: translating a uniform field element by $\alpha + U(\tau)$ or $\beta + V(\tau)$ preserves uniformity. Once A and B are fixed, the verification equation determines

$$C = \frac{AB - \alpha\beta - \gamma K_{\text{pub}}}{\delta}.$$

A simulator holding the setup trapdoor can therefore sample uniform A, B and compute this C without the witness. The resulting distribution is identical to that of an honest proof, which gives perfect zero knowledge for the original construction.

Soundness is subtler. Informally, the SRS permits the prover to form only prescribed linear combinations in the exponents. The α and β terms force the three proof elements to use a consistent assignment; the γ/δ separation prevents private coefficients from being substituted for verifier-controlled public inputs; and the hidden evaluation point makes a false polynomial identity hold only with negligible probability. Groth's analysis formalizes extraction against generic bilinear-group adversaries and establishes an argument of knowledge in that model [Gro16]. This assumption boundary should not be confused with an information-theoretic proof of knowledge in the standard model.

9.9 Protocol summary

Setup. Compile the fixed circuit to a QAP, sample $(\alpha, \beta, \gamma, \delta, \tau)$, publish the proving and verification keys, and destroy the trapdoor.

Prove. For a satisfying assignment, compute h, sample r, s, and output $\pi = ([A]_1, [B]_2, [C]_1)$.

Verify. Form $[K_{\text{pub}}]_1$ from the public input and check the single pairing-product equation above.

Cost. The proof contains three group elements. Verification uses three online pairings when $e([\alpha]_1, [\beta]_2)$ is precomputed; prover and key sizes scale with the circuit.

10 FRI: Fast Reed–Solomon IOPs of Proximity

The polynomial IOP chapters above repeatedly used the same verifier strategy: commit to a polynomial, evaluate it at a random point, and use Schwartz–Zippel to detect a false identity. This strategy presupposes that the committed oracle really is the evaluation of a *low-degree* polynomial. KZG enforces this through a structured reference string: the commitment can only be formed from the published powers up to the supported degree. A hash-based system has no such algebraic restriction. A Merkle root can bind the prover to an arbitrary table, including a table that is not close to any low-degree polynomial.

FRI fills precisely this gap [BSBHR18a]. Its full name is the Fast Reed–Solomon Interactive Oracle Proof of Proximity. It lets a verifier with oracle access to a large evaluation table test, using only a small number of queries, whether that table is close to a Reed–Solomon codeword. Its central operation resembles one layer of an inverse FFT: split a polynomial into its even and odd coefficient polynomials, combine those two pieces using a random verifier challenge, and recurse on a domain of half the size.

The main conceptual point is:

> Merkle commitment binds an arbitrary evaluation table,
> FRI proves that the table is close to a low-degree polynomial.

Neither component replaces the other. Together with Fiat–Shamir, they form the low-degree commitment layer used by many transparent proof systems.

10.1 The Reed–Solomon proximity problem

Let $D \subseteq \mathbb{F}$ be a set of N distinct points and let $1 \le d \le N$. Define the degree-$< d$ Reed–Solomon code on D by

$$\mathsf{RS}[\mathbb{F}, D, d] = \left\{ (p(x))_{x \in D} : p \in \mathbb{F}[X],\ \deg p < d \right\} \subseteq \mathbb{F}^D.$$

The dimension is d and the rate is

$$\rho = \frac{d}{N}.$$

The reciprocal

$$B = \rho^{-1} = \frac{N}{d}$$

is the *FRI blowup factor*. Starting from the d coefficients of a degree-$< d$ polynomial, the prover performs a low-degree extension to $N = Bd$ evaluations and commits only to those evaluations, rather than to all $|\mathbb{F}|$ field points. Thus B exposes the main engineering tradeoff:

- increasing B increases the prover's evaluation and hashing work, and the initial Merkle tree, essentially linearly;
- increasing B lowers the code rate and increases the distance, so fewer random query paths are usually needed for the same target soundness.

At the lecture-level heuristic where a far word has disagreement rate near $1 - \rho$, one query misses the disagreement with probability about $\rho = 1/B$. Hence t independent paths contribute at most roughly

$$t \log_2 B$$

bits of query soundness, suggesting $t \approx \lceil \lambda / \log_2 B \rceil$ for a λ-bit target. This is a parameter-selection heuristic, not a complete FRI security bound: finite-field terms, list-decoding radii, the exact FRI variant, and Fiat–Shamir grinding must also be included.

If p and q are distinct polynomials of degree less than d, then $p - q$ has at most $d - 1$ roots. Their evaluation vectors therefore disagree in at least $N - d + 1$ positions. Consequently, the relative minimum distance of the code is

$$\delta_{\min} = \frac{N - d + 1}{N} > 1 - \rho.$$

For two functions $f, g : D \to \mathbb{F}$, their relative Hamming distance is

$$\Delta_D(f, g) = \frac{1}{N} \left| \{ x \in D : f(x) \neq g(x) \} \right|.$$

The distance from f to the code is

$$\Delta_D(f, \mathsf{RS}[\mathbb{F}, D, d]) = \min_{\deg p < d} \Delta_D(f, p|_D).$$

Definition 8 (Reed–Solomon IOP of proximity)**.** *An interactive oracle proof of proximity for* $\mathsf{RS}[\mathbb{F}, D, d]$ *gives the verifier oracle access to a claimed word* $f^{(0)} : D \to \mathbb{F}$ *and to auxiliary prover oracles. It has:*

- ***perfect completeness*** *if every* $f^{(0)} \in \mathsf{RS}[\mathbb{F}, D, d]$ *is accepted by the honest verifier;*
- ***proximity soundness*** $s(\delta)$ *if every word at relative distance* δ *from the code is rejected with probability at least* $s(\delta)$*, regardless of the auxiliary oracles supplied by a cheating prover.*

The word *proximity* matters. A verifier making $q \ll N$ queries cannot reject every word that differs from a codeword in only one position: it may simply miss that position. FRI therefore gives a rejection probability that grows with the distance from the code. It is not a deterministic proof of exact code membership.

10.2 Smooth multiplicative domains

We use the multiplicative presentation most compatible with the preceding chapters. Let $\mathbb{F}$ have odd characteristic and suppose that $\mathbb{F}^\times$ contains a subgroup of order

$$N = 2^m.$$

Let ω generate that subgroup and let $\gamma \in \mathbb{F}^\times$. Define

$$D_0 = \gamma \langle \omega \rangle .$$

Successively square the domains:

$$D_i = \left\{ x^{2^i} : x \in D_0 \right\} = \gamma^{2^i} \left\langle \omega^{2^i} \right\rangle, \qquad |D_i| = \frac{N}{2^i}.$$

For every $i < m$, the squaring map

$$q_i : D_i \to D_{i+1}, \qquad q_i(x) = x^2,$$

is two-to-one. The two preimages of $y = x^2$ are x and $-x$.

In a prime field $\mathbb{F}_p$, a subgroup of order N exists exactly when $N \mid (p-1)$. This divisibility requirement explains the use of FFT-friendly fields. For example,

$$p = 2^{64} - 2^{32} + 1, \qquad p - 1 = 2^{32}(2^{32} - 1),$$

supports power-of-two subgroups through order 2^{32}. Write $\gamma_i = \gamma^{2^i}$, $\omega_i = \omega^{2^i}$, and $N_i = |D_i|$. Since N_i is even,

$$-\gamma_i \omega_i^j = \gamma_i \omega_i^{j+N_i/2}.$$

Thus the two entries paired by a binary fold have indices separated by $N_i/2$, and both map to the same point of D_{i+1}. Implementations commonly place the two values in the same Merkle leaf, or at least adjacent in the leaf ordering, so that opening a fold exposes and authenticates the pair together.

For clarity, assume first that the degree bound is a power of two,

$$d_0 = 2^r, \qquad r \le m,$$

and set

$$d_i = \frac{d_0}{2^i}.$$

Both the domain size and the degree bound halve in every round, so the code rate remains constant:

$$\frac{d_i}{|D_i|} = \frac{d_0}{N} = \rho.$$

After r rounds the alleged polynomial has degree less than 1, so it must be constant.

Remark 4 (Relation to the original FRI presentation)**.** *The formal main theorem of the original FRI paper is stated for additive cosets in binary extension fields. In characteristic two, x and $-x$ coincide, so the map $x \mapsto x^2$ is not the required two-to-one map; the construction instead uses affine-subspace polynomials, which are linearized polynomials. The same paper explains the smooth multiplicative-group version used here. Modern expositions over prime fields commonly use this roots-of-unity formulation because it exposes the FFT-style even/odd decomposition directly [BSBHR18a].*

10.3 One folding round

Let

$$p(X) = a_0 + a_1 X + \cdots + a_{d-1} X^{d-1}$$

have degree less than d. Separate its even and odd coefficients:

$$p_{\text{even}}(Y) = \sum_{j \geq 0} a_{2j} Y^j, \qquad p_{\text{odd}}(Y) = \sum_{j \geq 0} a_{2j+1} Y^j.$$

Then

$$\boxed{p(X) = p_{\text{even}}(X^2) + X\, p_{\text{odd}}(X^2).}$$

Both component polynomials have degree less than $\lceil d/2 \rceil$. Given a verifier challenge $\alpha \in \mathbb{F}$, define the folded polynomial

$$\text{Fold}_\alpha(p)(Y) = p_{\text{even}}(Y) + \alpha p_{\text{odd}}(Y).$$

Therefore

$$\deg \text{Fold}_\alpha(p) < \left\lceil \frac{d}{2} \right\rceil.$$

This is the source of FRI's degree reduction.

The verifier does not know the coefficients of p. It only has oracle access to values $f(x) = p(x)$ on D_i. Nevertheless, the folded value at $y = x^2$ can be recovered from the pair $x, -x$:

$$p(x) = p_{\text{even}}(y) + x p_{\text{odd}}(y),$$

$$p(-x) = p_{\text{even}}(y) - x p_{\text{odd}}(y).$$

Since the characteristic is odd,

$$p_{\text{even}}(y) = \frac{p(x) + p(-x)}{2},$$

$$p_{\text{odd}}(y) = \frac{p(x) - p(-x)}{2x}.$$

Hence the round-consistency equation is

$$\boxed{f^{(i+1)}(x^2) = \frac{f^{(i)}(x) + f^{(i)}(-x)}{2} + \alpha_i \frac{f^{(i)}(x) - f^{(i)}(-x)}{2x}.} \qquad \text{(FRI-fold)}$$

Equivalently, collecting the coefficients of the two opened values gives the Lagrange-weight form emphasized in many implementations:

$$\boxed{f^{(i+1)}(x^2) = \frac{\alpha_i + x}{2x} f^{(i)}(x) + \frac{\alpha_i - x}{-2x} f^{(i)}(-x).}$$

The two weights sum to one. As a useful sanity check, setting $\alpha_i = x$ returns $f^{(i)}(x)$, while setting $\alpha_i = -x$ returns $f^{(i)}(-x)$.

There is a useful interpolation interpretation. For fixed $y = x^2$, let $\ell_y(Z)$ be the unique affine polynomial satisfying

$$\ell_y(x) = f^{(i)}(x), \qquad \ell_y(-x) = f^{(i)}(-x).$$

Then

$$\ell_y(Z) = \frac{f^{(i)}(x) + f^{(i)}(-x)}{2} + Z\frac{f^{(i)}(x) - f^{(i)}(-x)}{2x},$$

and the folding check is simply

$$f^{(i+1)}(y) \stackrel{?}{=} \ell_y(\alpha_i).$$

Thus each fiber $\{x, -x\}$ defines a line, and the verifier asks the prover to supply the values of all those lines at one unpredictable horizontal coordinate α_i.

Why the challenge comes after the current oracle

If the prover could see α_i before binding itself to $f^{(i)}$, it could choose the two values in every pair so that their interpolating line passes through any desired $f^{(i+1)}(x^2)$. The order

$$\text{commit to } f^{(i)} \longrightarrow \alpha_i \longrightarrow \text{commit to } f^{(i+1)}$$

prevents this backwards construction. The random linear combination plays the same role as the verifier challenge in the sigma protocols studied earlier.

10.4 The pedagogical binary FRI protocol

We now state the complete arity-two protocol. The word $f^{(0)} : D_0 \to \mathbb{F}$ is claimed to be close to $\mathsf{RS}[\mathbb{F}, D_0, d_0]$, where $d_0 = 2^r$. In the IOP model, "send an oracle" means that the verifier may later query selected entries without reading the whole message.

Commit phase

The verifier begins with oracle access to $f^{(0)}$. For $i = 0, \ldots, r-1$:

1. the verifier samples a uniform challenge $\alpha_i \leftarrow \mathbb{F}$;
2. the prover sends a new oracle
$$f^{(i+1)} : D_{i+1} \to \mathbb{F},$$
allegedly satisfying the folding equation (FRI-fold) for every $x \in D_i$.

The final oracle $f^{(r)}$ is alleged to be constant. The prover sends that constant $c \in \mathbb{F}$. More generally, an implementation may stop earlier and send the coefficients of a small terminal polynomial, which the verifier evaluates directly.

The final domain has not generally collapsed to one point. Since $r = \log_2 d_0$,

$$|D_r| = \frac{N}{d_0} = B.$$

It is the *degree bound* that has collapsed to zero. Sending one field element c is therefore an assertion that all B remaining evaluations equal the same constant. If the protocol stops after $s < r$ folds, the terminal polynomial instead has degree $< d_0/2^s$; the prover sends its coefficients, and the verifier evaluates that polynomial at the terminal point reached by each query path.

The honest prover does not need to interpolate a polynomial in every round. For each pair $\{x, -x\}$, it computes one output value using (FRI-fold). The total number of field operations over all rounds is therefore a geometric sum:

$$O(N) + O(N/2) + O(N/4) + \cdots = O(N).$$

Query phase

After all oracles have been fixed, the verifier samples t independent query seeds

$$x_0^{(1)}, \ldots, x_0^{(t)} \leftarrow D_0.$$

For each path j set recursively

$$x_{i+1}^{(j)} = (x_i^{(j)})^2.$$

At round i, the verifier queries

$$f^{(i)}(x_i^{(j)}), \qquad f^{(i)}(-x_i^{(j)}), \qquad f^{(i+1)}(x_{i+1}^{(j)}),$$

and checks (FRI-fold). The value in the third query is reused as one of the two values in the next round. In the last round it is compared with the terminal constant c.

The verifier accepts only if every folding equation on every path is valid and the terminal degree check succeeds.

Commit-before-query discipline. The query locations must be sampled only after all oracles have been fixed. Otherwise the prover could make the checked paths consistent and place arbitrary values everywhere else. In the non-interactive version, this ordering is enforced by deriving every challenge and query seed from the transcript prefix that precedes it.

10.5 Perfect completeness

Lemma 2 (Degree reduction under folding)**.** *If* $\deg p_i < d_i$*, then for every* $\alpha_i \in \mathbb{F}$*,*

$$p_{i+1} = \mathrm{Fold}_{\alpha_i}(p_i)$$

satisfies

$$\deg p_{i+1} < \left\lceil \frac{d_i}{2} \right\rceil.$$

If d_i *is even, this is* $\deg p_{i+1} < d_i/2$*.*

Proof. Writing

$$p_i(X) = p_{i,\text{even}}(X^2) + X p_{i,\text{odd}}(X^2),$$

the largest exponent appearing in either component is obtained by dividing an exponent of p_i by two and rounding down. Thus both component degrees are less than $\lceil d_i/2 \rceil$. Their linear combination has no larger degree. □

Theorem 1 (Perfect completeness of binary folding)**.** *Suppose $f^{(0)} = p_0|_{D_0}$ for some polynomial p_0 of degree less than $d_0 = 2^r$. If the prover sets*

$$p_{i+1} = \text{Fold}_{\alpha_i}(p_i), \qquad f^{(i+1)} = p_{i+1}|_{D_{i+1}},$$

then every verifier check passes, and p_r is constant.

Proof. The folding equation follows from the even/odd decomposition, so every local query check is an identity. By the lemma and induction,

$$\deg p_i < \frac{d_0}{2^i}.$$

At $i = r$ this gives $\deg p_r < 1$, hence p_r is constant and equals the value sent by the honest prover. □

10.6 Why local consistency implies proximity soundness

Completeness is elementary; soundness is the deep part of FRI. A local folding check only shows consistency among three queried values. A cheating prover may choose each oracle to be an arbitrary function, not the evaluation of any polynomial. The analysis must show that a word far from the initial Reed–Solomon code cannot be threaded through all folding rounds while making most local checks pass.

For fixed oracles and a fixed challenge, define the round inconsistency rate

$$\varepsilon_i = \Pr_{x \leftarrow D_i}\left[f^{(i+1)}(x^2) \neq \frac{f^{(i)}(x) + f^{(i)}(-x)}{2} + \alpha_i \frac{f^{(i)}(x) - f^{(i)}(-x)}{2x} \right].$$

If ε_i is large, a random query catches the inconsistency directly. If it is small, then $f^{(i+1)}$ agrees on most points with the random fold of $f^{(i)}$. The crucial FRI proximity-gap theorem says, roughly, that random folding does not usually turn a word that is far from a Reed–Solomon code into one that is much closer to the smaller code.

The reason randomness is necessary can be seen from list decoding. A received word may be close to several low-degree candidates. Each candidate pair induces algebraic conditions on α_i under which their folded images collide or become unusually close. Because the list of relevant candidates is controlled and α_i is sampled after the current oracle is fixed, only a small set of challenges is bad. Outside that set, either distance survives in the next round or the prover creates noticeable local inconsistency.

The simplified security calculation presented in many lectures separates two failure modes as

$$\Pr[\text{accept}] \lesssim \varepsilon_{\text{field}} + (1 - \delta)^t.$$

Here $\varepsilon_{\text{field}}$ accounts for exceptional folding challenges—in a basic Schwartz–Zippel-style discussion it is on the order of a degree bound divided by $|\mathbb{F}|$—and $(1-\delta)^t$ is the probability that all t query paths miss a δ fraction of bad locations. This formula is valuable intuition, but it is not a substitute for the theorem of the particular FRI variant and proximity regime being instantiated.

There is also a matching interpolation attack explaining why the rate cannot be ignored. Assume for simplicity that d is even. A cheating prover can choose a union T of $d/2$ pairs $\{x, -x\} \subset D_0$, so $|T| = d = \rho N$, and interpolate the unique degree-$< d$ polynomial s agreeing with the initial word on T. It then builds every later oracle by honestly folding s, not the full initial word. A query path whose first pair lies in T sees a perfectly consistent transcript; a uniformly sampled pair lies in T with probability ρ. Consequently, all t paths pass with probability

$$\rho^t = 2^{-t \log_2(1/\rho)}.$$

This attack works even when the initial word is otherwise very far from the code. It shows that no analysis can credit a binary-FRI path with more than about $\log_2(1/\rho)$ bits merely from sampling. It also explains why a larger blowup factor improves query efficiency.

The original FRI theorem makes this quantitative for Reed–Solomon codes of rate $\rho = 2^{-R}$, $R \geq 2$. For a word at distance δ from the code, its optimized IOPP rejects with probability at least

$$\min\{\delta(1 - o(1)), \delta_0\},$$

where $\delta_0 > 0$ depends mainly on the code rate. For block length N, the construction has linear prover arithmetic, logarithmic verifier arithmetic, and logarithmic query complexity [BSBHR18a]. The precise theorem uses the additive-domain, higher-arity protocol rather than the pedagogical binary version above, but the same proximity-preserving mechanism is at work.

The many-round structure also calls for a stronger, round-by-round view of soundness. Later work proved round-by-round soundness and knowledge soundness for FRI and Batched FRI, and used those properties to justify a non-interactive random-oracle compilation [BGK+23]. A representative bound in the regime $\delta < 1 - \sqrt{\rho}$ has the schematic form

$$\varepsilon_{\text{rbr}} = \max\left\{ O\left(\frac{N^2}{\rho^{3/2}|\mathbb{F}|}\right), (1-\delta)^t \right\},$$

with exact notation and constants depending on the protocol. Subsequent concrete analysis shows that conjectured and provable parameter estimates can differ substantially, so real systems should report the theorem or conjecture under which their claimed bit security is computed [BT24].

If one execution rejects a δ-far word with probability at least $s(\delta)$, then t independent query paths reduce the conditional acceptance probability to at most

$$(1 - s(\delta))^t.$$

Concrete systems choose the code rate, folding arity, field, and number of queries together; one should not estimate security from the number of queries alone.

10.7 Compiling the oracle protocol with Merkle trees

The IOP model gives the verifier ideal oracle access. A cryptographic proof realizes each oracle using a Merkle commitment.

Commit phase with hashes

For each evaluation vector

$$\mathbf{f}^{(i)} = \left(f^{(i)}(x)\right)_{x \in D_i},$$

the prover builds a Merkle tree and publishes its root R_i. The transcript order is

$$\begin{aligned} R_0 \longrightarrow \alpha_0 &= H(\mathsf{fri\text{-}challenge}, R_0) \longrightarrow R_1 \\ &\longrightarrow \alpha_1 = H(\mathsf{fri\text{-}challenge}, R_0, R_1) \longrightarrow R_2 \longrightarrow \cdots . \end{aligned}$$

The hash outputs are mapped into field elements. Domain-separation labels and the complete statement must be included in the real transcript.

Collision resistance gives computational binding: after publishing R_i, the prover cannot open one position to two different field values without producing a hash collision. This is the hash-based analogue of the binding role played by a PCS commitment.

Grinding and Fiat–Shamir security accounting

Replacing each verifier message by a hash does more than remove the network interaction: it lets the prover evaluate the random oracle repeatedly before publishing a proof. If one complete interactive transcript accepts falsely with probability ε, then an adversary trying Q independently varied transcript prefixes has the basic grinding opportunity suggested by $Q\varepsilon$. For a κ-bit random oracle, a representative classical-ROM accounting for FRI has the form [BGK+23]

$$\varepsilon_{\mathrm{FS}} \leq Q\,\varepsilon_{\mathrm{rbr}} + O\left(\frac{Q^2}{2^\kappa}\right).$$

Thus "λ bits interactively" does not mean that an implementation may ignore the attacker's hashing budget. For example, 2^b offline trials can consume roughly b bits from a naive $2^{-\lambda}$ target.

For an arbitrary many-round public-coin protocol, standard soundness alone need not prevent a stronger attack that grinds one round at a time, retaining each favorable prefix. This is why the lecture stresses *round-by-round soundness*: from any doomed transcript prefix, the next random challenge should leave the state doomed except with small probability. The lecture slides predate the cited proof of this property for FRI; the correct modern conclusion is therefore not that Fiat–Shamir FRI lacks a proof, but that non-interactive security relies on the dedicated round-by-round/BCS analysis and its concrete parameters, rather than on the slogan "replace every challenge by a hash."

Query phase with authentication paths

After the final root or terminal polynomial has been sent, Fiat–Shamir derives the query indices from the full transcript. For every queried value, the prover supplies a Merkle authentication path. The verifier:

1. reconstructs each root and checks that every opened value belongs to the previously committed oracle;
2. checks the FRI folding equations along each path;
3. checks the terminal polynomial and its degree bound.

The IOP query complexity of binary FRI is $O(t \log d_0)$ field elements. With separate binary Merkle paths, the authentication data along one path is naively $O(\log^2 N)$ hash values, because the proof opens leaves in a sequence of progressively smaller trees. Batched openings, Merkle multiproofs, larger folding arity, and larger tree arity reduce the concrete overhead substantially.

10.8 FRI is a low-degree test, not by itself a PCS

It is common to say "FRI commitment," but three layers should be distinguished:

Layer	Responsibility
Merkle tree	bind to an evaluation table
FRI IOPP	prove proximity of the table to a low-degree RS codeword
evaluation reduction	connect a claimed point value to that codeword

For example, to prove $p(z) = v$, define

$$q(X) = \frac{p(X) - v}{X - z}.$$

If the claim is correct and $\deg p < d$, then q is a polynomial of degree less than $d - 1$. On an evaluation domain D not containing z, the verifier can check at sampled points that

$$p(x) - v = (x - z)q(x),$$

while FRI proves that the committed q-table is close to a polynomial of the required degree. In the simple unique-decoding interpretation, the distance must remain below roughly half the relative minimum distance, about $(1 - \rho)/2$, so that there is a unique nearby polynomial to which the evaluation claim can refer. In that regime the elementary miss probability per query is larger than ρ and may contribute less than one bit per path, even when the raw ρ^t attack suggests $\log_2(1/\rho)$ bits. This is one reason FRI is often best understood as providing *list/proximity binding*: the Merkle root and FRI transcript constrain the prover to a small set of nearby low-degree candidates, and the surrounding evaluation reduction must select the candidate consistent with $p(z) = v$. More refined protocols sample outside the evaluation domain to bind the prover more tightly to a candidate polynomial. DEEP-FRI formalizes this "domain extending for eliminating pretenders" idea and improves the soundness range while retaining linear prover arithmetic and logarithmic verifier arithmetic [BSGKS20].

10.9 Where FRI sits inside a STARK

FRI does not prove arbitrary computation directly. In a STARK-style system the layers are:

execution trace $\longrightarrow$ AIR constraints
$\longrightarrow$ composition/quotient polynomial
$\longrightarrow$ low-degree extension on a large domain
$\longrightarrow$ FRI proximity proof
$\longrightarrow$ Merkle commitments + Fiat–Shamir.

AIR and the algebraic linking layer establish that the committed polynomial encodes a valid execution if it is low degree. FRI supplies that final low-degree guarantee. Merkle trees make the polynomial oracles succinctly accessible, and Fiat–Shamir removes interaction. This is the hash-based route to scalable transparent arguments described by the STARK construction [BSBHR18b].

Remark 5 (Transparency and post-quantum security)**.** *FRI requires no hidden algebraic trapdoor. Once compiled with Merkle trees, its cryptographic assumption is principally the security of the hash function, which is why FRI-based systems are described as transparent and plausibly post-quantum. Concrete post-quantum parameters must still account for quantum attacks on the hash function and for the security analysis of Fiat–Shamir in the quantum random-oracle model.*

Remark 6 (Zero knowledge is an additional layer)**.** *Plain FRI is not automatically zero knowledge. Its query answers reveal evaluations of the committed codeword, and a Merkle root is binding but not an information-theoretic hiding commitment. A zero-knowledge STARK randomizes or masks the underlying polynomial oracles and proves that the masking preserves the required identities. FRI then proves low degree of the masked objects. Thus transparency, proximity soundness, and zero knowledge are separate properties that must each be established.*

10.10 Complexity and design tradeoffs

For the binary pedagogical protocol with t query paths and $r = \log_2 d_0$ folds:

Quantity	Asymptotic cost
folding arithmetic for the prover	$O(N)$
total committed field elements	$< 2N$
verifier field queries	$O(t \log d_0)$
verifier field arithmetic	$O(t \log d_0)$
cryptographic setup	transparent; public hash function
proof size after Merkle compilation	polylogarithmic, but larger than KZG proofs

Higher-arity FRI decomposes

$$p(X) = \sum_{j=0}^{b-1} X^j p_j(X^b)$$

and folds the b coefficient polynomials using powers of a random challenge. It reduces the number of rounds by a factor of $\log_2 b$ but opens more sibling values per round. This is a typical proof-system tradeoff: larger arity shortens the recursion while increasing the local work and the width of each query.

The enduring reason FRI is useful is not that any one cost is minimal. KZG has much smaller openings and faster verification. FRI instead combines:

- linear-time algebraic folding,
- logarithmic verifier arithmetic and query complexity,
- no trusted setup,
- hash-based commitments, and
- compatibility with large FFT-style execution traces.

Appendix D works through every fold and one complete authenticated query path over a small field.

11 Synthesis and Outlook

The constructions in this note can now be organized along two largely independent axes:

$$\boxed{\text{arithmetization and oracle protocol}} \quad + \quad \boxed{\text{commitment and compilation layer}}.$$

R1CS/QAP and Plonkish constraint systems choose different ways to express computation. KZG, IPA, pairings, and FRI choose different ways to bind the prover to algebraic data and verify it succinctly. These choices determine whether a system needs a trusted setup, whether its security is pairing-, discrete-log-, or hash-based, and whether proof size and verification are constant, logarithmic, or polylogarithmic.

Across these systems, the recurring proof pattern is:

1. encode a computation and witness as algebraic objects;
2. reduce global correctness to a small collection of polynomial relations;
3. bind the prover to the relevant polynomials or evaluation tables;
4. sample unpredictable challenges and test only a few evaluations;
5. compile the public-coin interaction into a non-interactive argument, typically with Fiat–Shamir.

Groth16 realizes this pattern with a circuit-specific pairing SRS and a three-element proof. Plonk separates a reusable polynomial IOP from the PCS used to instantiate it. FRI replaces algebraic-group commitments with Merkle commitments and recursive low-degree testing, giving transparency and post-quantum-oriented assumptions at the cost of larger proofs. The appendices work through concrete instances of these abstractions and make the indexing, interpolation, transcript, and folding calculations explicit.

A A Complete Bulletproofs-Style Polynomial Opening Example

This appendix gives a concrete exponent-level example of the Bulletproofs/IPA-style folding idea [BBB+18]. The example is not cryptographically secure because the field is tiny; it is meant to make the algebra visible.

A.1 The polynomial opening claim

Work over

$$\mathbb{F}_{97}.$$

Let

$$f(X) = 2 + 4X + X^2 + 3X^3.$$

The coefficient vector is

$$\mathbf{f} = (2, 4, 1, 3).$$

Let the evaluation point be

$$u = 5.$$

Then

$$\begin{aligned} v = f(5) &= 2 + 4 \cdot 5 + 5^2 + 3 \cdot 5^3 \\ &= 2 + 20 + 25 + 375 \\ &= 422 \\ &\equiv 34 \pmod{97}. \end{aligned}$$

Equivalently,

$$v = \langle (2, 4, 1, 3), (1, 5, 25, 28) \rangle = 34,$$

since $5^3 = 125 \equiv 28 \pmod{97}$.

A.2 Pedersen vector commitment

Let

$$\mathsf{pp} = (g_0, g_1, g_2, g_3)$$

be four independent group bases in a cyclic group of order 97. The commitment is

$$\mathsf{com}_f = g_0^2 g_1^4 g_2^1 g_3^3.$$

The prover wants to prove that this committed vector evaluates to 34 at $u = 5$.

A.3 First folding round

Split the polynomial into low and high halves:

$$\mathbf{f}_L = (2, 4), \qquad \mathbf{f}_R = (1, 3).$$

Compute

$$\begin{aligned} v_L &= 2 + 4 \cdot 5 = 22, \\ v_R &= 1 + 3 \cdot 5 = 16. \end{aligned}$$

Then

$$v_L + u^2 v_R = 22 + 25 \cdot 16 = 422 \equiv 34 = v.$$

The prover sends

$$L_1 = g_2^2 g_3^4, \qquad R_1 = g_0^1 g_1^3.$$

Let the verifier challenge be

$$r_1 = 7, \qquad r_1^{-1} = 14 \pmod{97}.$$

Fold the coefficients:

$$\begin{aligned} f'_0 &= r_1 f_0 + f_2 = 7 \cdot 2 + 1 = 15, \\ f'_1 &= r_1 f_1 + f_3 = 7 \cdot 4 + 3 = 31. \end{aligned}$$

So

$$\mathbf{f}' = (15, 31).$$

Fold the bases:

$$g'_0 = g_0^{14} g_2, \qquad g'_1 = g_1^{14} g_3.$$

Fold the value:

$$v' = r_1 v_L + v_R = 7 \cdot 22 + 16 = 170 \equiv 73 \pmod{97}.$$

Indeed,

$$f'(5) = 15 + 31 \cdot 5 = 170 \equiv 73.$$

Now check the commitment update:

$$\mathsf{com}' = L_1^7 \mathsf{com}_f R_1^{14}.$$

Expanding:

$$\begin{aligned}\mathsf{com}' &= (g_2^2 g_3^4)^7 (g_0^2 g_1^4 g_2 g_3^3)(g_0 g_1^3)^{14} \\ &= g_0^{2+14} g_1^{4+42} g_2^{14+1} g_3^{28+3} \\ &= g_0^{16} g_1^{46} g_2^{15} g_3^{31}.\end{aligned}$$

On the other hand,

$$\begin{aligned}(g_0')^{15}(g_1')^{31} &= (g_0^{14} g_2)^{15}(g_1^{14} g_3)^{31} \\ &= g_0^{210} g_2^{15} g_1^{434} g_3^{31} \\ &= g_0^{16} g_1^{46} g_2^{15} g_3^{31} \pmod{97},\end{aligned}$$

because $210 \equiv 16$ and $434 \equiv 46$ modulo 97. Thus the folded commitment is correct.

A.4 Second folding round and final scalar

The remaining vector has length 2. Split

$$\mathbf{f}' = (15, 31)$$

into left scalar 15 and right scalar 31. The verifier first checks the value decomposition

$$v' = 15 + u \cdot 31 = 15 + 5 \cdot 31 = 170 \equiv 73.$$

The prover sends

$$L_2 = (g_1')^{15}, \qquad R_2 = (g_0')^{31}.$$

Let the second challenge be

$$r_2 = 11, \qquad r_2^{-1} = 53 \pmod{97}.$$

The final folded scalar is

$$f'' = r_2 \cdot 15 + 31 = 196 \equiv 2 \pmod{97}.$$

The final folded base is

$$g'' = (g_0')^{53} g_1'.$$

The second commitment update is

$$\mathsf{com}'' = L_2^{r_2} \mathsf{com}' R_2^{r_2^{-1}}.$$

Expanding with $\mathsf{com}' = (g_0')^{15}(g_1')^{31}$ gives

$$\begin{aligned}\mathsf{com}'' &= ((g_1')^{15})^{11}(g_0')^{15}(g_1')^{31}((g_0')^{31})^{53} \\ &= (g_0')^{15+31\cdot 53}(g_1')^{31+15\cdot 11}.\end{aligned}$$

Modulo 97,

$$15 + 31 \cdot 53 = 1658 \equiv 9, \qquad 31 + 15 \cdot 11 = 196 \equiv 2.$$

On the other hand,

$$\begin{aligned}(g'')^{f''} &= ((g_0')^{53} g_1')^2 \\ &= (g_0')^{106} (g_1')^2 \\ &= (g_0')^9 (g_1')^2,\end{aligned}$$

since $106 \equiv 9 \pmod{97}$. Thus

$$\mathsf{com}'' = (g'')^{f''},$$

so the commitment relation survives the second folding round as well.

This is the whole logic of Bulletproofs-style opening: a long vector commitment and inner-product claim are recursively folded into a scalar claim. The proof length grows only logarithmically in the original vector length.

B A Complete Plonk Example over $\mathbb{F}_{97}$

This appendix gives a complete Plonk-style proof sketch at the arithmetization and IOP-gadget level. It includes ProductCheck inside the wiring/permutation argument. It is not a production Plonk transcript with every batching, blinding, and Fiat–Shamir challenge-ordering detail spelled out.

B.1 Field and subgroup

Work over the prime field

$$\mathbb{F}_{97}.$$

We use a subgroup of size 12 because the trace occupies $3|C| + |I| = 12$ slots. The element

$$\omega = 6$$

has order 12 in $\mathbb{F}_{97}^{\times}$. Hence

$$\Omega = \left\{1, \omega, \omega^2, \ldots, \omega^{11}\right\}$$

with actual values

$$\Omega = \{1, 6, 36, 22, 35, 16, 96, 91, 61, 75, 62, 81\}.$$

The vanishing polynomial is

$$Z_\Omega(X) = X^{12} - 1.$$

B.2 Circuit and statement

Take the three-gate computation

$$(x_1 + x_2)(x_2 + w_1).$$

Let

$$x_1 = 5, \qquad x_2 = 6, \qquad w_1 = 1.$$

Then

$$(5 + 6)(6 + 1) = 11 \cdot 7 = 77.$$

The computation trace is:

Gate	L	R	O	Type
0	5	6	11	$+$
1	6	1	7	$+$
2	11	7	77	$\times$

B.3 Trace polynomial

We place public and witness inputs at negative powers and gate wires at consecutive triples:

Exponent i	ω^i	Meaning	$T(\omega^i)$
11	81	x_1	5
10	62	x_2	6
9	75	w_1	1
0	1	gate 0 left	5
1	6	gate 0 right	6
2	36	gate 0 output	11
3	22	gate 1 left	6
4	35	gate 1 right	1
5	16	gate 1 output	7
6	96	gate 2 left	11
7	91	gate 2 right	7
8	61	gate 2 output	77

There is a unique polynomial of degree at most 11 interpolating these values. It is

$$\begin{aligned} T(X) = 20 &+ 85X + 22X^2 + 20X^3 + 80X^4 + 10X^5 \\ &+ 47X^6 + 38X^7 + 27X^8 + 70X^9 + 6X^{10} + 65X^{11} \pmod{97}. \end{aligned}$$

B.4 Public input check

The verifier knows $x_1 = 5$ and $x_2 = 6$. Define $v(X)$ by

$$v(\omega^{11}) = 5, \qquad v(\omega^{10}) = 6.$$

The interpolating degree-1 polynomial is

$$v(X) = 45 + 51X \pmod{97}.$$

The public input condition is

$$T(y) - v(y) = 0 \qquad \forall y \in \Omega_{\text{inp}} = \left\{\omega^{11}, \omega^{10}\right\}.$$

Indeed,

$$T(81) = 5 = v(81), \qquad T(62) = 6 = v(62).$$

This would be proved by ZeroTest on the two-point input domain.

B.5 Gate check

The gate-start domain is

$$\Omega_{\text{gates}} = \left\{\omega^0, \omega^3, \omega^6\right\} = \{1, 22, 96\}.$$

Define a selector polynomial S by

$$S(1) = 1, \qquad S(22) = 1, \qquad S(96) = 0.$$

The degree-2 interpolant is

$$S(X) = 68 + 49X + 78X^2 \pmod{97}.$$

The gate polynomial is

$$G_{\text{gate}}(X) = S(X)(T(X) + T(\omega X)) + (1 - S(X))T(X)T(\omega X) - T(\omega^2 X).$$

We need

$$G_{\text{gate}}(y) = 0 \qquad \forall y \in \Omega_{\text{gates}}.$$

Concretely:

Gate-start y	Check
1	$5 + 6 - 11 = 0$
$22 = \omega^3$	$6 + 1 - 7 = 0$
$96 = \omega^6$	$11 \cdot 7 - 77 = 0$

This condition is also proved by a ZeroTest, now over Ω_{gates}.

B.6 Wiring as a prescribed permutation

The copy constraints are:

$$T(\omega^{10}) = T(\omega^1) = T(\omega^3) = 6,$$
$$T(\omega^{11}) = T(\omega^0) = 5,$$
$$T(\omega^2) = T(\omega^6) = 11,$$
$$T(\omega^9) = T(\omega^4) = 1.$$

The permutation $W : \Omega \to \Omega$ acts by cycles

$$(\omega^{10}\ \omega^1\ \omega^3)(\omega^{11}\ \omega^0)(\omega^2\ \omega^6)(\omega^9\ \omega^4),$$

with $\omega^5, \omega^7, \omega^8$ fixed. The logical wiring condition is

$$T(y) = T(W(y)) \qquad \forall y \in \Omega.$$

Interpolating W on Ω gives one polynomial representative

$$\begin{aligned} W(X) = {} & 53X + 38X^2 + 50X^3 + 8X^4 + 81X^5 + 43X^6 \\ & + 80X^7 + 94X^8 + 44X^9 + 21X^{10} + 54X^{11} \pmod{97}. \end{aligned}$$

A direct check of $T(X) - T(W(X))$ would involve a high-degree composition: the polynomial $W(X)$ is merely an interpolation of the wiring table, not a cheap function to compose into T. Plonk therefore uses a prescribed permutation ProductCheck.

B.7 ProductCheck inside the wiring argument

The prescribed permutation check samples random challenges $\rho, \sigma \in \mathbb{F}_{97}$ and proves

$$\prod_{a \in \Omega} \frac{\rho - \sigma W(a) - T(a)}{\rho - \sigma a - T(a)} = 1.$$

For this concrete example choose

$$\rho = 17, \qquad \sigma = 3.$$

Define

$$F(a) = \rho - \sigma W(a) - T(a), \qquad G(a) = \rho - \sigma a - T(a),$$

and

$$h(a) = \frac{F(a)}{G(a)}.$$

The following table lists the domain order and the running product values

$$t(\omega^s) = \prod_{i=0}^{s} h(\omega^i).$$

i	$a = \omega^i$	$T(a)$	$W(a)$	$h(a)$	$t(a)$
0	1	5	81	39	39
1	6	6	22	91	57
2	36	11	96	37	72
3	22	6	62	12	88
4	35	1	75	83	29
5	16	7	16	1	29
6	96	11	36	21	27
7	91	7	91	1	27
8	61	77	61	1	27
9	75	1	35	90	5
10	62	6	6	66	39
11	81	5	1	5	1

The final value is

$$t(\omega^{11}) = 1,$$

so the product identity holds. Notice that the entries with $h(a) = 1$ correspond to fixed points of the wiring permutation. The interpolated running product polynomial is

$$\begin{aligned} t(X) = {} & 69 + 25X + 27X^2 + 18X^3 + 79X^4 + 64X^5 \\ & + 83X^6 + 17X^7 + 13X^8 + 88X^9 + 53X^{10} + 85X^{11} \pmod{97}. \end{aligned}$$

Now the prover interpolates the running product polynomial $t(X)$ from the values above. The recurrence to be checked is the rational ProductCheck recurrence:

$$t(\omega X)G(\omega X) = t(X)F(\omega X) \qquad \forall X \in \Omega.$$

Equivalently, define

$$R(X) = t(\omega X)G(\omega X) - t(X)F(\omega X).$$

Then

$$R(X) = 0 \qquad \forall X \in \Omega,$$

so

$$q(X) = \frac{R(X)}{X^{12} - 1}$$

is a polynomial. A verifier checks this quotient relation at one random point z.

For example, take

$$z = 2.$$

The interpolated polynomials give

$$R(2) = 25, \qquad z^{12} - 1 = 2^{12} - 1 \equiv 21 \pmod{97}.$$

Since

$$21^{-1} \equiv 37 \pmod{97},$$

we get

$$q(2) = R(2)(z^{12} - 1)^{-1} = 25 \cdot 37 \equiv 52 \pmod{97}.$$

Thus the verifier's random-point check is

$$R(2) \stackrel{?}{=} q(2)(2^{12} - 1),$$

and indeed

$$52 \cdot 21 = 1092 \equiv 25 \pmod{97}.$$

This is the ProductCheck embedded inside the Plonk wiring proof. The table proves the randomized pair-multiset equality. By the prescribed permutation lemma, this implies $T(a) = T(W(a))$ for every wire slot $a \in \Omega$, except with the usual random-challenge soundness error.

B.8 Output check

In this toy trace, the final output slot is

$$T(\omega^8) = 77.$$

If the statement is "the circuit output is 77," the verifier asks for a direct opening proving

$$T(\omega^8) = 77.$$

If the proof system uses a zero-output convention, one appends a final linear gate computing

$$T(\omega^8) - 77$$

and checks that the new final output is zero.

B.9 The final proof obligations

The complete Plonk proof for this toy circuit consists of the following obligations:

1. **Trace commitment:** the prover commits to T.
2. **Public inputs:** $T - v$ vanishes on Ω_{inp}.
3. **Gate semantics:** G_{gate} vanishes on Ω_{gates}.
4. **Wiring:** the prescribed permutation relation is proved by the rational ProductCheck above.
5. **Output:** the final output equals 77 or equals zero after a final adjustment gate.

Each obligation is a low-degree polynomial identity checked at one or a few random points. A PCS such as KZG or Bulletproofs turns these oracle checks into a succinct argument [KZG10, BBB+18]. This is the complete architecture of Plonk in miniature.

C A Color-Coded R1CS-to-QAP Example

This appendix gives a complete worked example of the transformation

$$\text{algebraic circuit} \longrightarrow \text{R1CS} \longrightarrow \text{QAP}.$$

The example is intentionally color coded. The colors are a reading device for tracking how one object is represented in the next layer. This is the arithmetization route underlying QAP-based systems such as GGPR, Pinocchio, and Groth16 [GGPR13, PHGR13, Gro16]. We stop at the QAP divisibility relation and do not construct a Groth16 proof.

C.1 Color legend: wires, rows, and basis polynomials

There are two independent color systems.

How to read the colors.

- **Wire colors track columns.** The same wire has the same color in the algebraic circuit, in the assignment vector, in the R1CS matrix column, and in the QAP column polynomial.
- **Constraint colors track rows.** The same constraint color is used for the algebraic constraint $\mathcal{C}_i$, the R1CS row i, the constraint point r_i, and the Lagrange basis polynomial $L_i(X)$.
- In the QAP formula
$$A_j(X) = \sum_{i=1}^{m} A_{i,j} L_i(X),$$
the column index j is the wire being encoded, while the basis polynomial $L_i(X)$ remembers which R1CS row supplied the coefficient.

The wire-color legend is

wire	1	x_1	x_2	w_1	v_1	v_2	y
role	constant	input	input	witness	intermediate	intermediate	output

The constraint-color legend is

$$\mathcal{C}_1 \leftrightarrow r_1 = 1 \leftrightarrow L_1(X), \qquad \mathcal{C}_2 \leftrightarrow r_2 = 2 \leftrightarrow L_2(X), \qquad \mathcal{C}_3 \leftrightarrow r_3 = 3 \leftrightarrow L_3(X).$$

Thus a term such as $L_2(X)$ inside $A_{x_2}(X)$ should be read as: "the x_2 column of the A matrix has a nonzero coefficient in row $\mathcal{C}_2$."

C.2 The algebraic circuit

Work over a field $\mathbb{F}$ of characteristic not equal to 2 or 3; for concrete numerical checks, one may read all values in $\mathbb{F}_{97}$. Consider the circuit

$$y = (x_1 + x_2)(x_2 + w_1).$$

Flatten the computation by introducing two intermediate wires:

$$\begin{aligned} \mathcal{C}_1 : &\quad (x_1 + x_2) \cdot 1 = v_1, \\ \mathcal{C}_2 : &\quad (x_2 + w_1) \cdot 1 = v_2, \\ \mathcal{C}_3 : &\quad v_1 \cdot v_2 = y. \end{aligned}$$

These three equations already have the R1CS shape

$$(\text{linear form}) \cdot (\text{linear form}) = (\text{linear form}).$$

The concrete assignment used below is

$$x_1 = 5, \qquad x_2 = 6, \qquad w_1 = 1,$$

so that

$$v_1 = 11, \qquad v_2 = 7, \qquad y = 77.$$

The full assignment vector is

$$z = (1, x_1, x_2, w_1, v_1, v_2, y) = (1, 5, 6, 1, 11, 7, 77).$$

C.3 R1CS matrices with row and column labels

Using the column order

$$(1, x_1, x_2, w_1, v_1, v_2, y),$$

the A matrix encodes the left linear form in each constraint:

A	1	x_1	x_2	w_1	v_1	v_2	y
$\mathcal{C}_1$	0	1	1	0	0	0	0
$\mathcal{C}_2$	0	0	1	1	0	0	0
$\mathcal{C}_3$	0	0	0	0	1	0	0

Read the entry $A_{2,w_1} = 1$ as follows: it is in the w_1 column, so it is the coefficient of w_1; it is in the $\mathcal{C}_2$ row, so it belongs to the second constraint.

The B matrix encodes the right linear form in each constraint:

B	1	x_1	x_2	w_1	v_1	v_2	y
$\mathcal{C}_1$	1	0	0	0	0	0	0
$\mathcal{C}_2$	1	0	0	0	0	0	0
$\mathcal{C}_3$	0	0	0	0	0	1	0

The first two constraints multiply by the constant wire 1; the third constraint multiplies by v_2.

The C matrix encodes the output linear form in each constraint:

C	1	x_1	x_2	w_1	v_1	v_2	y
$\mathcal{C}_1$	0	0	0	0	1	0	0
$\mathcal{C}_2$	0	0	0	0	0	1	0
$\mathcal{C}_3$	0	0	0	0	0	0	1

The row colors show which equation a coefficient belongs to; the column colors show which wire the coefficient multiplies.

For the assignment $z = (1, 5, 6, 1, 11, 7, 77)$, the three rows check as follows:

$$(A_1 \cdot z)(B_1 \cdot z) = (5 + 6) \cdot 1 = 11 = C_1 \cdot z,$$

$$(A_2 \cdot z)(B_2 \cdot z) = (6 + 1) \cdot 1 = 7 = C_2 \cdot z,$$

$$(A_3 \cdot z)(B_3 \cdot z) = 11 \cdot 7 = 77 = C_3 \cdot z.$$

Thus this assignment satisfies the R1CS.

C.4 From R1CS entries to QAP basis terms

Choose the constraint domain

$$H = \{r_1 = 1, r_2 = 2, r_3 = 3\}.$$

The vanishing polynomial is

$$Z_H(X) = (X - 1)(X - 2)(X - 3) = X^3 - 6X^2 + 11X - 6.$$

The Lagrange basis polynomials for H are

$$L_1(X) = \frac{(X - 2)(X - 3)}{(1 - 2)(1 - 3)} = \frac{(X - 2)(X - 3)}{2},$$

$$L_2(X) = \frac{(X-1)(X-3)}{(2-1)(2-3)} = -(X-1)(X-3),$$
$$L_3(X) = \frac{(X-1)(X-2)}{(3-1)(3-2)} = \frac{(X-1)(X-2)}{2}.$$

They satisfy $L_i(r_j) = 1$ if $i = j$ and 0 otherwise.

The column-interpolation rule is

$$A_j(X) = \sum_{i=1}^{3} A_{i,j}\, L_i(X), \qquad B_j(X) = \sum_{i=1}^{3} B_{i,j}\, L_i(X), \qquad C_j(X) = \sum_{i=1}^{3} C_{i,j}\, L_i(X).$$

Hence an R1CS number becomes a QAP coefficient: the entry $A_{i,j}$ becomes the coefficient of the row-colored basis polynomial $L_i(X)$ inside the wire-colored column polynomial $A_j(X)$.

For example, in the A matrix the nonzero entries become:

R1CS entry	source	QAP contribution
$A_{1,x_1} = 1$	x_1 appears in $\mathcal{C}_1$	$1 \cdot L_1(X)$
$A_{1,x_2} = 1$	x_2 appears in $\mathcal{C}_1$	$1 \cdot L_1(X)$
$A_{2,x_2} = 1$	x_2 appears in $\mathcal{C}_2$	$1 \cdot L_2(X)$
$A_{2,w_1} = 1$	w_1 appears in $\mathcal{C}_2$	$1 \cdot L_2(X)$
$A_{3,v_1} = 1$	v_1 appears in $\mathcal{C}_3$	$1 \cdot L_3(X)$

This table is the key bridge. It shows exactly how a colored number in the R1CS matrix turns into a colored basis term in the QAP.

C.5 Constructing the QAP column polynomials

Now collect all contributions for each wire column.

For the A matrix:

wire column	$A_j(X)$
x_1	$L_1(X)$
x_2	$L_1(X) + L_2(X)$
w_1	$L_2(X)$
v_1	$L_3(X)$

All other $A_j(X)$ are zero. The term $L_2(X)$ inside $A_{x_2}(X)$ is precisely the QAP image of the R1CS entry $A_{2,x_2} = 1$.

For the B matrix:

wire column	$B_j(X)$
1	$L_1(X) + L_2(X)$
v_2	$L_3(X)$

The constant wire appears in the first two B-linear forms, while v_2 appears in the third.

For the C matrix:

wire column	$C_j(X)$
v_1	$L_1(X)$
v_2	$L_2(X)$
y	$L_3(X)$

For instance, $C_y(X) = L_3(X)$ says that y is the output wire of constraint $\mathcal{C}_3$.

C.6 Combining columns with the assignment

The assignment values turn the column polynomials into three aggregate polynomials:

$$A_z(X) = \sum_j z_j A_j(X), \qquad B_z(X) = \sum_j z_j B_j(X), \qquad C_z(X) = \sum_j z_j C_j(X).$$

Using the colored columns above,

$$\begin{aligned} A_z(X) &= 5\,L_1(X) + 6(L_1(X) + L_2(X)) + 1\,L_2(X) + 11\,L_3(X) \\ &= 11L_1(X) + 7L_2(X) + 11L_3(X) \\ &= 4X^2 - 16X + 23, \end{aligned}$$

$$\begin{aligned} B_z(X) &= 1(L_1(X) + L_2(X)) + 7\,L_3(X) \\ &= L_1(X) + L_2(X) + 7L_3(X) \\ &= 3X^2 - 9X + 7, \end{aligned}$$

$$\begin{aligned} C_z(X) &= 11\,L_1(X) + 7\,L_2(X) + 77\,L_3(X) \\ &= 11L_1(X) + 7L_2(X) + 77L_3(X) \\ &= 37X^2 - 115X + 89. \end{aligned}$$

The color of each L_i term tells which R1CS row generated that contribution. Evaluating at the row-colored points reproduces the row values:

X	$A_z(X)$	$B_z(X)$	$C_z(X)$
1	11	1	11
2	7	1	7
3	11	7	77

Thus at each constraint point,

$$A_z(r_i)B_z(r_i) = C_z(r_i).$$

C.7 Checking the QAP divisibility condition

Define the QAP numerator

$$P_z(X) = A_z(X)B_z(X) - C_z(X).$$

Using the explicit polynomials above,

$$\begin{aligned} P_z(X) &= (4X^2 - 16X + 23)(3X^2 - 9X + 7) - (37X^2 - 115X + 89) \\ &= 12X^4 - 84X^3 + 204X^2 - 204X + 72. \end{aligned}$$

Direct evaluation gives

$$P_z(1) = P_z(2) = P_z(3) = 0.$$

Therefore P_z is divisible by

$$Z_H(X) = X^3 - 6X^2 + 11X - 6.$$

Indeed,

$$P_z(X) = (12X - 12)Z_H(X),$$

so the quotient polynomial is

$$h_z(X) = 12X - 12.$$

The final QAP relation for this example is

$$\boxed{A_z(X)B_z(X) - C_z(X) = h_z(X)Z_H(X).}$$

The cross-representation trace is now visible:

$$\begin{array}{c}\text{colored algebraic constraint } \mathcal{C}_i \\ \Updownarrow \\ \text{colored R1CS row } i \\ \Updownarrow \\ \text{colored QAP basis polynomial } L_i(X).\end{array}$$

This is exactly the algebraic statement that the next Groth16 layer will commit to and check using pairings.

D A Complete FRI Folding Example over $\mathbb{F}_{17}$

This appendix gives a complete arithmetic transcript for binary multiplicative-domain FRI protocol of Section 10. The field and domain are far too small for cryptographic security. Their purpose is to make every evaluation, fold, terminal degree check, and Merkle opening location explicit.

D.1 Parameters and evaluation domains

Work over

$$\mathbb{F} = \mathbb{F}_{17}.$$

The element

$$\omega = 3$$

has order 16 in $\mathbb{F}_{17}^{\times}$. Take the initial domain

$$D_0 = \langle \omega \rangle = (1, 3, 9, 10, 13, 5, 15, 11, 16, 14, 8, 7, 4, 12, 2, 6).$$

The order shown here is important: it is the leaf order used by the first Merkle tree. Squaring gives

$$D_1 = \left\langle \omega^2 \right\rangle = \langle 9 \rangle = (1, 9, 13, 15, 16, 8, 4, 2),$$

and squaring once more gives

$$D_2 = \left\langle \omega^4 \right\rangle = \langle 13 \rangle = (1, 13, 16, 4).$$

The sizes are

$$|D_0| = 16, \qquad |D_1| = 8, \qquad |D_2| = 4.$$

We prove proximity to the Reed–Solomon code

$$\mathsf{RS}[\mathbb{F}_{17}, D_0, 4].$$

Thus the claimed degree bound is

$$d_0 = 4$$

and the code rate is

$$\rho = \frac{4}{16} = \frac{1}{4}.$$

Each fold halves both the domain size and the degree bound:

round i	$\lvert D_i\rvert$	d_i	$d_i/\lvert D_i\rvert$
0	16	4	1/4
1	8	2	1/4
2	4	1	1/4

After two folds, a degree-< 1 polynomial must be constant.

D.2 The initial Reed–Solomon word

Let the initial polynomial be

$$p_0(X) = 3 + 5X + 2X^2 + 7X^3.$$

It has degree $3 < 4$, so its evaluation word is a valid codeword. Evaluating in the domain order above gives

$$\begin{aligned}\mathbf{f}^{(0)} &= \left(p_0(x)\right)_{x\in D_0}\\ &= (0, 4, 9, 11, 9, 1, 13, 12, 10, 4, 15, 4, 10, 3, 9, 2).\end{aligned}$$

For example,

$$\begin{aligned}p_0(10) &= 3 + 5\cdot 10 + 2\cdot 10^2 + 7\cdot 10^3\\ &= 7253\\ &\equiv 11 \pmod{17}.\end{aligned}$$

The points halfway around the subgroup differ by a sign because

$$\omega^8 = 16 = -1 \pmod{17}.$$

Thus the two preimages under squaring are paired by indices j and $j + 8$:

x	$-x$	x^2
1	16	1
3	14	9
9	8	13
10	7	15
13	4	16
5	12	8
15	2	4
11	6	2

D.3 First folding round

Split p_0 into even and odd coefficient polynomials:

$$\begin{aligned}p_0(X) &= (3 + 2X^2) + X(5 + 7X^2)\\ &= p_{0,\mathrm{even}}(X^2) + X p_{0,\mathrm{odd}}(X^2),\end{aligned}$$

where

$$p_{0,\mathrm{even}}(Y) = 3 + 2Y, \qquad p_{0,\mathrm{odd}}(Y) = 5 + 7Y.$$

Let the first verifier challenge be

$$\alpha_0 = 4.$$

The folded polynomial is

$$\begin{aligned} p_1(Y) &= p_{0,\text{even}}(Y) + \alpha_0 p_{0,\text{odd}}(Y) \\ &= (3 + 2Y) + 4(5 + 7Y) \\ &= 23 + 30Y \\ &\equiv 6 + 13Y \pmod{17}. \end{aligned}$$

As required,

$$\deg p_1 = 1 < 2 = \frac{d_0}{2}.$$

We now reproduce this fold using only evaluation values. Since

$$2^{-1} \equiv 9 \pmod{17},$$

define for each pair $\{x, -x\}$

$$E_0(x^2) = \frac{f^{(0)}(x) + f^{(0)}(-x)}{2},$$

$$O_0(x^2) = \frac{f^{(0)}(x) - f^{(0)}(-x)}{2x}.$$

The complete first-round table is:

x	$-x$	$y = x^2$	$f^{(0)}(x)$	$f^{(0)}(-x)$	$E_0(y)$	$O_0(y)$	$E_0(y) + 4O_0(y)$
1	16	1	0	10	5	12	2
3	14	9	4	4	4	0	4
9	8	13	9	15	12	11	5
10	7	15	11	4	16	8	14
13	4	16	9	10	1	15	10
5	12	8	1	3	2	10	8
15	2	4	13	9	11	16	7
11	6	2	12	2	7	2	15

These folded values are exactly the evaluations of $p_1(Y) = 6 + 13Y$ on D_1:

$$\begin{aligned} \mathbf{f}^{(1)} &= \big(p_1(y)\big)_{y \in D_1} \\ &= (2, 4, 5, 14, 10, 8, 7, 15). \end{aligned}$$

As one nontrivial row check, take $x = 9$, $-x = 8$, and $y = x^2 = 13$. We have

$$f^{(0)}(9) = 9, \qquad f^{(0)}(8) = 15.$$

The even component is

$$E_0(13) = \frac{9 + 15}{2} = 24 \cdot 9 \equiv 12,$$

and, because

$$(2 \cdot 9)^{-1} = 1^{-1} = 1 \pmod{17},$$

the odd component is

$$O_0(13) = \frac{9 - 15}{2 \cdot 9} \equiv -6 \equiv 11.$$

Therefore

$$E_0(13) + 4O_0(13) = 12 + 4 \cdot 11 = 56 \equiv 5,$$

which agrees with

$$p_1(13) = 6 + 13 \cdot 13 = 175 \equiv 5.$$

D.4 Second folding round

The degree-1 polynomial p_1 already has the decomposition

$$p_1(Y) = 6 + 13Y = p_{1,\text{even}}(Y^2) + Y p_{1,\text{odd}}(Y^2),$$

where

$$p_{1,\text{even}}(Z) = 6, \qquad p_{1,\text{odd}}(Z) = 13.$$

Let the second verifier challenge be

$$\alpha_1 = 6.$$

Then

$$\begin{aligned} p_2(Z) &= p_{1,\text{even}}(Z) + \alpha_1 p_{1,\text{odd}}(Z) \\ &= 6 + 6 \cdot 13 \\ &= 84 \\ &\equiv 16 \pmod{17}. \end{aligned}$$

Thus p_2 is the constant polynomial 16, as required by the terminal degree bound $d_2 = 1$.

The complete evaluation-level fold is:

y	$-y$	$z = y^2$	$f^{(1)}(y)$	$f^{(1)}(-y)$	$E_1(z)$	$O_1(z)$	$E_1(z) + 6O_1(z)$
1	16	1	2	10	6	13	16
9	8	13	4	8	6	13	16
13	4	16	5	7	6	13	16
15	2	4	14	15	6	13	16

Therefore

$$\mathbf{f}^{(2)} = \big(p_2(z)\big)_{z \in D_2} = (16, 16, 16, 16).$$

In the terminal step the prover need not commit to four identical values. It can send the single coefficient

$$c_2 = 16,$$

and the verifier treats the terminal oracle as the constant function $f^{(2)}(z) = c_2$.

D.5 A concrete commit transcript

We now place the arithmetic tables inside the cryptographic transcript. Let $\mathsf{MerkleRoot}$ denote a binary Merkle-tree root, with the leaf index included in the hashed leaf encoding.

The prover first commits to the initial evaluation vector:

$$R_0 = \mathsf{MerkleRoot}\big(\mathbf{f}^{(0)}\big).$$

The verifier sends $\alpha_0 = 4$. In a non-interactive proof this value would be derived as

$$\alpha_0 = \mathsf{HashToField}\big(\text{fri-alpha-0} \,\|\, R_0\big).$$

For this toy transcript, we stipulate that the resulting field element is 4.

The prover computes and commits to the first folded vector:

$$R_1 = \mathsf{MerkleRoot}\big(\mathbf{f}^{(1)}\big).$$

The second challenge is

$$\alpha_1 = \mathsf{HashToField}\big(\text{fri-alpha-1} \,\|\, R_0 \,\|\, R_1\big) = 6.$$

The prover computes the terminal polynomial and sends

$$c_2 = 16.$$

Only now is the query index derived:

$$\begin{aligned} j &= \mathsf{HashToIndex}(\mathsf{fri\text{-}query} \,\|\, R_0 \,\|\, R_1 \,\|\, c_2) \\ &= 0. \end{aligned}$$

The equalities to 4, 6, and 0 are representative Fiat–Shamir outputs chosen to make the arithmetic transcript concrete. They are not claims about a particular deployed hash function.

D.6 One complete query path

The query index $j = 0$ selects

$$x_0 = \omega^0 = 1 \in D_0.$$

The path of squared points is

$$x_0 = 1, \qquad x_1 = x_0^2 = 1 \in D_1, \qquad x_2 = x_1^2 = 1 \in D_2.$$

Round 0 check

The verifier opens

$$f^{(0)}(1) = 0, \qquad f^{(0)}(-1) = f^{(0)}(16) = 10,$$

under R_0, and

$$f^{(1)}(1) = 2$$

under R_1. The locally reconstructed even value is

$$\frac{0 + 10}{2} = 10 \cdot 9 = 90 \equiv 5,$$

and the odd value is

$$\frac{0 - 10}{2 \cdot 1} = 7 \cdot 9 = 63 \equiv 12.$$

The FRI equation gives

$$\begin{aligned} \frac{f^{(0)}(1) + f^{(0)}(16)}{2} + \alpha_0 \frac{f^{(0)}(1) - f^{(0)}(16)}{2} &= 5 + 4 \cdot 12 \\ &= 53 \\ &\equiv 2 \\ &= f^{(1)}(1). \end{aligned}$$

The first round accepts.

Round 1 check

The already opened value $f^{(1)}(1) = 2$ is reused. The verifier additionally opens

$$f^{(1)}(-1) = f^{(1)}(16) = 10$$

under R_1. The even and odd values are

$$\frac{2+10}{2} = 12 \cdot 9 = 108 \equiv 6,$$

$$\frac{2-10}{2 \cdot 1} = 9 \cdot 9 = 81 \equiv 13.$$

Using $\alpha_1 = 6$,

$$\begin{aligned} \frac{f^{(1)}(1) + f^{(1)}(16)}{2} + \alpha_1 \frac{f^{(1)}(1) - f^{(1)}(16)}{2} &= 6 + 6 \cdot 13 \\ &= 84 \\ &\equiv 16 \\ &= c_2. \end{aligned}$$

The second round accepts, and c_2 describes a degree-< 1 terminal polynomial. This completes the arithmetic verification path.

D.7 The Merkle authentication paths

To make the oracle realization explicit, let $L_k^{(i)}$ denote leaf k of the tree for $\mathbf{f}^{(i)}$, and let $N_{a:b}^{(i)}$ denote the internal node covering the contiguous leaf interval $a, \ldots, b$.

In the first tree, the round-0 query opens indices

$$0 \quad \text{and} \quad 8,$$

corresponding to the points 1 and $16 = -1$. A separate authentication path for index 0 would contain

$$L_1^{(0)}, \quad N_{2:3}^{(0)}, \quad N_{4:7}^{(0)}, \quad N_{8:15}^{(0)}.$$

A separate path for index 8 would contain

$$L_9^{(0)}, \quad N_{10:11}^{(0)}, \quad N_{12:15}^{(0)}, \quad N_{0:7}^{(0)}.$$

When the two openings are batched, $N_{0:7}^{(0)}$ and $N_{8:15}^{(0)}$ are reconstructed from the opened branches. The minimal sibling-node set is therefore

$$\left\{ L_1^{(0)}, N_{2:3}^{(0)}, N_{4:7}^{(0)}, L_9^{(0)}, N_{10:11}^{(0)}, N_{12:15}^{(0)} \right\}.$$

Together with the opened leaf values 0 and 10, these nodes reconstruct R_0.

In the second tree, the full path needs indices

$$0 \quad \text{and} \quad 4,$$

corresponding to 1 and $16 = -1$ in the D_1 ordering. The batched sibling-node set is

$$\left\{ L_1^{(1)}, N_{2:3}^{(1)}, L_5^{(1)}, N_{6:7}^{(1)} \right\}.$$

Together with the opened values 2 and 10, these nodes reconstruct R_1.

The verifier rejects before doing field arithmetic if either reconstructed root differs from the committed root. Merkle verification therefore binds the numerical inputs used in both folding equations to the oracles fixed during the commit phase.

D.8 How a corrupted value is detected

Suppose, for illustration, that the first entry were changed from

$$f^{(0)}(1) = 0$$

to

$$\widetilde{f}^{(0)}(1) = 1,$$

while the queried values $f^{(0)}(16) = 10$ and $f^{(1)}(1) = 2$ remained unchanged. Under the same challenge $\alpha_0 = 4$, the verifier would reconstruct

$$\widetilde{E}_0(1) = \frac{1 + 10}{2} = 11 \cdot 9 = 99 \equiv 14,$$

$$\widetilde{O}_0(1) = \frac{1 - 10}{2} = 8 \cdot 9 = 72 \equiv 4.$$

The expected folded value would become

$$14 + 4 \cdot 4 = 30 \equiv 13,$$

but the committed next-round value is 2. Hence

$$13 \neq 2,$$

and this query path rejects.

This calculation illustrates local consistency detection. It is not by itself a soundness proof: a malicious prover is free to alter later oracles as well. The global FRI theorem is what shows that if the initial word is far from every degree-< 4 codeword, then random challenges and random query paths force either a detectable local inconsistency or a terminal word that fails the final degree check.

D.9 The complete proof obligations

For this example the verifier's obligations are:

1. accept R_0 only as a commitment to a length-16 word indexed by D_0;
2. derive α_0 after R_0 is fixed;
3. accept R_1 only as a commitment to a length-8 word indexed by D_1;
4. derive α_1 after R_1 is fixed;
5. receive the terminal constant $c_2 = 16$;
6. derive query indices only after the complete commit transcript is fixed;
7. verify every Merkle authentication path;
8. check the round-0 folding equation;
9. check the round-1 folding equation against c_2;
10. check that the terminal description has degree less than 1.

The worked path verifies one randomly selected chain. A cryptographic instance repeats the query phase at many independent indices and uses much larger fields and domains. The arithmetic, however, is exactly the same as in the two equations checked above.

References

[ALM+98] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998.

[AS98] Sanjeev Arora and Shmuel Safra. Probabilistic checking of proofs: A new characterization of NP. *Journal of the ACM*, 45(1):70–122, 1998.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE, 2018.

[BBD+23] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In *Advances in Cryptology — CRYPTO 2023*, volume 14085 of *Lecture Notes in Computer Science*. Springer, 2023.

[BCC+16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Advances in Cryptology—EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 327–357. Springer, 2016.

[BDL+11] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In *Cryptographic Hardware and Embedded Systems — CHES 2011*, volume 6917 of *Lecture Notes in Computer Science*, pages 124–142. Springer, 2011.

[BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, pages 103–112. ACM, 1988.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In *Advances in Cryptology—EUROCRYPT 2020*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, 2020.

[BG12] Stephanie Bayer and Jens Groth. Efficient zero-knowledge argument for correctness of a shuffle. In *Advances in Cryptology—EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 263–280. Springer, 2012.

[BGG+88] Michael Ben-Or, Oded Goldreich, Shafi Goldwasser, Johan Håstad, Joe Kilian, Silvio Micali, and Phillip Rogaway. Everything provable is provable in zero-knowledge. In *Advances in Cryptology — CRYPTO 1988*, volume 403 of *Lecture Notes in Computer Science*, pages 37–56. Springer, 1988.

[BGK+23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat–shamir security of FRI and related SNARKs. Cryptology ePrint Archive, Paper 2023/1071, 2023.

[BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *Advances in Cryptology—ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 514–532. Springer, 2001.

[BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics*, pages 14:1–14:17. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018.

[BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.

[BSCG+13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for c: Verifying program executions succinctly and in zero knowledge. Cryptology ePrint Archive, Paper 2013/507, 2013.

[BSCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 781–796. USENIX Association, 2014.

[BSGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Report 2019/336, 2020.

[BT24] Alexander R. Block and Pratyush Ranjan Tiwari. On the concrete security of non-interactive FRI. Cryptology ePrint Archive, Paper 2024/1161, 2024.

[CDG+17] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1825–1842. ACM, 2017.

[DL78] Richard A. DeMillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Information Processing Letters*, 7(4):193–195, 1978.

[FHK+20] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-Fourier lattice-based compact signatures over NTRU. NIST Post-Quantum Cryptography Round 3 Finalist Specification, 2020.

[For87] Lance Fortnow. The complexity of perfect zero-knowledge. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing (STOC)*, pages 204–209. ACM, 1987.

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology—CRYPTO '86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1987.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In *Advances in Cryptology—EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 626–645. Springer, 2013.

[GK03] Shafi Goldwasser and Yael Tauman Kalai. On the (In)security of the Fiat–Shamir paradigm. In *44th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 102–113. IEEE, 2003.

[GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[GMW86] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity and a methodology of cryptographic protocol design. In *27th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 174–187. IEEE, 1986.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology—EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.

[GW20] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, Paper 2020/315, 2020.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.

[IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing (STOC)*, pages 21–30. ACM, 2007.

[Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments. In *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*, pages 723–732. ACM, 1992.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology—ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010.

[Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of Cryptography—TCC 2021*, volume 13043 of *Lecture Notes in Computer Science*, pages 1–34. Springer, 2021.

[MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2111–2128. ACM, 2019.

[Ped92] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology—CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1992.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE, 2013.

[PST13] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In *Theory of Cryptography—TCC 2013*, volume 7785 of *Lecture Notes in Computer Science*, pages 222–242. Springer, 2013.

[Sch80] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980.

[Sch89] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology — CRYPTO 1989*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252. Springer, 1989.

[Sha90] Adi Shamir. IP = PSPACE. In *31st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 11–15. IEEE, 1990.

[WTS+18] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE, 2018.

[Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation: EUROSAM '79*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.